Prototipo de sistema de traducción de Lengua de Señas Argentina con procesamiento en tiempo cuasi real

Maite Nigro Francisco Veron

Director: Esp. Ing. Gonzalo Avalos Ribas

Co-director: Ing. Bruno Constanzo

Universidad Nacional de Mar del Plata
Facultad de Ingeniería
Departamento de Ingeniería en Informática

Marzo 2026

Agradecimientos

De Maite: Para la Flaqui.
De Francisco: Para el Diego y Baco.
A Miriam Rolls, por su orientación sobre la LSA y la validación del vocabulario del conjunto de datos.

Índice general

1. Introducción **1**
1.1. Contexto . . . 1
1.2. Motivación . . . 1
1.3. Problemática . . . 1
1.4. Alcance y limitaciones . . . 2
1.4.1. Delimitación del vocabulario y asesoramiento lingüístico . . . 3
1.4.2. Límites operativos y comportamiento ante entradas no contempladas 4
1.4.3. Infraestructura de despliegue . . . 4

2. Objetivos **5**
2.1. Objetivo general . . . 5
2.2. Objetivos específicos . . . 5

3. Marco teórico, estado del arte y metodología **7**
3.1. Marco teórico . . . 7
3.1.1. Componentes articulatorios de la LSA . . . 7
3.1.2. Estructura lingüística de la LSA y traducción *Topic-Comment* . . . 7
3.1.3. Visión computacional y reconocimiento basado en esqueletos . . . 8
3.1.4. Arquitectura neuronal temporal: TinySkeletonClassifier . . . 10
3.1.5. Modelos de lenguaje pequeños y adaptación LoRA . . . 12
3.1.6. Animación y representación visual mediante Avatar . . . 13
3.1.7. Arquitectura híbrida de partición Cliente-Nube . . . 14
3.1.8. Captura de medios en la Web y aislamiento en extensiones (Manifest V3) . . . 14
3.2. Estado del arte . . . 15
3.2.1. Desarrollos en Argentina y la región . . . 15
3.2.2. Sistemas internacionales de reconocimiento y síntesis . . . 15
3.3. Metodología de desarrollo . . . 16
3.3.1. Declaración sobre el uso de herramientas de inteligencia artificial . . 16
3.4. Herramientas y tecnologías empleadas . . . 18

4. Análisis de requerimientos **19**
4.1. Requerimientos funcionales . . . 19
4.2. Requerimientos no funcionales . . . 20
4.3. Restricciones . . . 20
4.3.1. Especificación del entorno de ejecución cliente . . . 21
4.4. Casos de uso . . . 22
4.4.1. Casos de uso esenciales en formato breve . . . 22
4.4.2. Casos de uso expandidos . . . 22

5. Diseño del Sistema **27**
5.1. Arquitectura del sistema . . . 27
5.1.1. Justificación de la partición y modelo de privacidad . . . 27
5.1.2. Flujo integral de datos entre componentes. . . . 27
5.2. Diseño de contratos de API REST . . . 31
5.2.1. API local del motor ILSA . . . 31
5.2.2. API del servicio semántico remoto . . . 33
5.2.3. API del servicio de archivos del avatar . . . 34
5.3. Diseño de la representación gestual inversa . . . 35
5.3.1. Clip de referencia . . . 36
5.3.2. Preparación y empalme . . . 36
5.3.3. Presentación en la videollamada . . . 38
5.3.4. Contingencia léxica . . . 39
5.4. Patrones de diseño de software aplicados . . . 40
5.5. Diseño conceptual de la arquitectura escalable en la nube . . . 41
5.5.1. Especificación para entornos de producción concurrente . . . 41
5.5.2. Instancia de prueba en Cloud Run . . . 42
5.5.3. Servicio de archivos del avatar . . . 42

6. Implementación y Desarrollo **44**
6.1. Implementación del subsistema de reconocimiento gestual . . . 44
6.1.1. Preprocesamiento de datos . . . 44
6.1.2. Preprocesamiento y arquitectura neuronal TinySkeletonClassifier . . . 44
6.2. Implementación del subsistema de traducción semántica bidireccional . . . 45
6.2.1. Selección y especificación de la suite de modelos de lenguaje . . . 45
6.2.2. Canal directo (LSA → Español) . . . 45
6.2.3. Canal inverso (Español → LSA) . . . 47
6.2.4. Tratamiento de letras y dígitos sueltos . . . 49
6.2.5. Cuantización GGUF Q4_K_M y servicio de inferencia . . . 49
6.3. Integración en Google Meet y extensión de navegador . . . 50
6.3.1. Despliegue en Chrome Manifest V3: aislamiento offscreen y sandbox 51
6.3.2. Intercepción de la captura de cámara y lienzo virtual . . . 51
6.3.3. Motor local ILSA (FastAPI) . . . 52
6.4. Implementación del avatar . . . 52
6.4.1. Tecnologías empleadas . . . 52
6.4.2. Vía descartada: retarget hacia un avatar VRM . . . 53
6.4.3. Estructura del pedido . . . 54
6.4.4. Decisiones técnicas de la reproducción . . . 54
6.5. Instancia de validación del prototipo mediante túnel seguro . . . 55

7. Experimentación, Resultados y Discusión **56**
7.1. Corpus y metodología de adquisición de Datos de LSA . . . 56
7.1.1. Definición y validación del vocabulario de 100 clases . . . 56
7.1.2. Protocolo de grabación y curación del dataset propio . . . 56
7.2. Evaluación empírica del clasificador gestual . . . 57
7.2.1. Métricas offline vs. rendimiento en cámara real . . . 57
7.2.2. Impacto de la resolución temporal: 16 vs. 32 fotogramas . . . 58
7.2.3. Búsqueda automatizada de hiperparámetros con Optuna y Análisis de sobreajuste . . . 59

7.2.4. Diagnóstico de confusiones mediante feature engineering y separabilidad . . . 60
7.2.5. Impacto cuantitativo de la corrección de aspect ratio . . . 61
7.2.6. Desempeño final del clasificador baseline en vivo . . . 61
7.3. Evaluación de los módulos de traducción semántica . . . 64
7.3.1. Benchmark comparativo de modelos SLM (Familias Qwen2.5, Llama 3.2 y SmolLM2) . . . 64
7.3.2. Evaluación cuantitativa: BLEU, ROUGE-L, METEOR y exactitud 64
7.3.3. Análisis de simetría en el aprendizaje: traducción directa vs. estructuración inversa . . . 64
7.3.4. Compromiso entre calidad lingüística y latencia de inferencia . . . 64
7.4. Evaluación del sistema integrado y rendimiento en tiempo real . . . 64
7.4.1. Validación funcional de extremo a extremo en videollamadas reales 64
7.4.2. Análisis de latencias totales por etapa del pipeline . . . 64
7.4.3. Consumo de recursos de hardware: máquina de desarrollo vs. máquina objetivo . . . 64
7.4.4. Discusión técnica y lecciones de ingeniería aprendidas . . . 64

8. Conclusiones y trabajo futuro 65
8.1. Conclusiones . . . 65
8.1.1. Cumplimiento de objetivos del proyecto . . . 65
8.1.2. Aportes técnicos y de accesibilidad . . . 65
8.1.3. Impacto social en el marco de la Ley 27.710 . . . 65
8.2. Trabajo futuro . . . 65
8.2.1. Implementación física del clúster semántico escalable . . . 65
8.2.2. Expansión del corpus multiseñante y mitigación de variabilidad . . 65
8.2.3. Incorporación de rasgos no manuales (expresiones faciales) . . . 65
8.2.4. Generalización a nuevas plataformas de comunicación virtual . . . 65

Resumen

En la actualidad, la interacción virtual entre personas sordas y oyentes se limita casi exclusivamente a la comunicación escrita o a la transcripción unilateral de voz a texto, lo que genera una marcada asimetría en la comunicación expresiva durante videollamadas. Este Trabajo Final de Grado desarrolla un prototipo de traducción bidireccional entre la Lengua de Señas Argentina (LSA) y el español oral estructurado bajo una arquitectura híbrida de computación en el borde y servicios en la nube con procesamiento en tiempo pseudo-real.

El sistema articula dos canales comunicacionales complementarios. En el sentido señante-oyente, la computadora del usuario captura localmente el flujo de video desde la cámara web, extrae coordenadas articulares tridimensionales mediante visión computacional y clasifica un vocabulario de 100 señas mediante una red neuronal ligera (Conv1D + Transformer con *attention pooling*). Con el fin de evitar la saturación de memoria y cómputo que provocaría ejecutar modelos de lenguaje generativos en dispositivos convencionales, las glosas clasificadas son enviadas hacia un backend semántico en la nube que transforma la secuencia en oraciones fluidas en español rioplatense, las cuales retornan al cliente para su proyección en subtítulos o síntesis de voz. En sentido inverso (oyente-señante), el sistema procesa la voz del interlocutor oyente mediante un módulo semántico inverso en la nube que estructura las glosas, enviándolas a un avatar tridimensional en el cliente que ejecuta las animaciones correspondientes o recurre al deletreo dactilológico en términos fuera del léxico.

El módulo clasificador gestual local alcanza un 98,28 % de exactitud top-1 en validación fuera de línea y un 77,3 % de acierto top-1 (94,8 % top-3) en cámara real. Por su parte, los modelos de lenguaje evaluados (Qwen2.5, Llama 3.2 y SmolLM adaptados mediante LoRA y cuantizados a GGUF Q4_K_M) superan el 94 % de exactitud y alcanzan latencias de inferencia de entre 200 y 450 ms. Como mecanismo de integración para videollamadas, el sistema se acopla a Google Meet mediante una extensión de navegador conectada al motor local ILSA, asegurando la privacidad del usuario al no transmitir video crudo fuera del dispositivo cliente. Finalmente, se deja documentada la propuesta de diseño de una arquitectura en la nube escalable para soportar la demanda concurrente en entornos de producción.

Palabras clave:

Lengua de Señas Argentina, reconocimiento de gestos, arquitectura híbrida cliente-nube, traducción semántica bidireccional, avatar 3D, accesibilidad.

Capítulo 1

Introducción

1.1. Contexto

Tradicionalmente, la sociedad abordó a la comunidad sorda desde una perspectiva clínica centrada en la rehabilitación y la corrección auditiva. Ese enfoque omite que la comunidad sorda se identifica, ante todo, como una minoría lingüística con cultura y gramática propias, independientes del español oral. En la Argentina, la Lengua de Señas Argentina (LSA) cuenta con reconocimiento legal formal a través de la Ley 27.710 (República Argentina, 2023), la cual garantiza su preservación y el derecho de las personas sordas a comunicarse en su lengua natural en todos los ámbitos sociales.

En el contexto digital contemporáneo, las plataformas de videoconferencia se han convertido en herramientas centrales para el ámbito laboral, educativo y de la vida cotidiana. Sin embargo, los entornos de software de uso masivo presentan escasa integración nativa de herramientas bilingües automatizadas que admitan la LSA como un flujo comunicacional bidireccional en tiempo pseudo-real. Los recursos tecnológicos existentes suelen restringirse a diccionarios estáticos o repositorios de video, carentes de capacidad de interpretación contextual e interactiva durante una llamada.

1.2. Motivación

Este proyecto nace de la aspiración de reducir las barreras comunicacionales de las personas señantes en interacciones cotidianas y de asistencia mediante herramientas informáticas. Resulta indispensable avanzar hacia esquemas de integración lingüística en los que la tecnología actúe como un puente transparente en ambos sentidos de la conversación: expresivo (señante $\rightarrow$ oyente) y receptivo (oyente $\rightarrow$ señante).

1.3. Problemática

La problemática central radica en la asimetría comunicacional presente en las plataformas de videoconferencia. Si bien las herramientas de transcripción automática de voz a texto han alcanzado un alto grado de adopción, resultan insuficientes como solución de accesibilidad integral: para diversas personas sordas, el texto escrito opera como una segunda lengua con reglas gramaticales ajenas, e incluso genera barreras de acceso para quienes no se encuentran alfabetizados en la lengua oral.

En el extremo expresivo, la comunicación mediante LSA carece de soporte nativo en el software de videollamadas comercial. La falta de módulos capaces de capturar e interpretar la LSA hacia voz o texto audible fuerza la dependencia de interpretación humana presencial o restringe la interacción al chat escrito.

A esta brecha se añade la especificidad lingüística local: los modelos de visión y lenguaje desarrollados internacionalmente se encuentran enfocados mayoritariamente en la Lengua de Señas Americana (ASL). Las técnicas y arquitecturas neuronales resultan reutilizables, pero los modelos preentrenados y conjuntos de datos de ASL no se extrapolan directamente a la LSA debido a diferencias léxicas, dactilológicas y sintácticas sustanciales, por lo que es necesaria la construcción de un corpus propio y de modelos adaptados al contexto argentino.

1.4. Alcance y limitaciones

El sistema desarrollado comprende un prototipo funcional de traducción bidireccional en tiempo pseudo-real concebido como una extensión para navegadores basados en la tecnología Chromium (como Google Chrome, Microsoft Edge, Brave, Opera o Vivaldi), articulada con un motor de ejecución en segundo plano denominado ILSA (*Intérprete de LSA*).

Para abordar la interpretación de la LSA se introduce el concepto de **glosa**: representación escrita en letras mayúsculas asignada a una seña para identificarla y analizar su gramática sin constituir una traducción literal término a término. La arquitectura general y el flujo funcional de bloques del prototipo se esquematizan en la Figura 1.1.

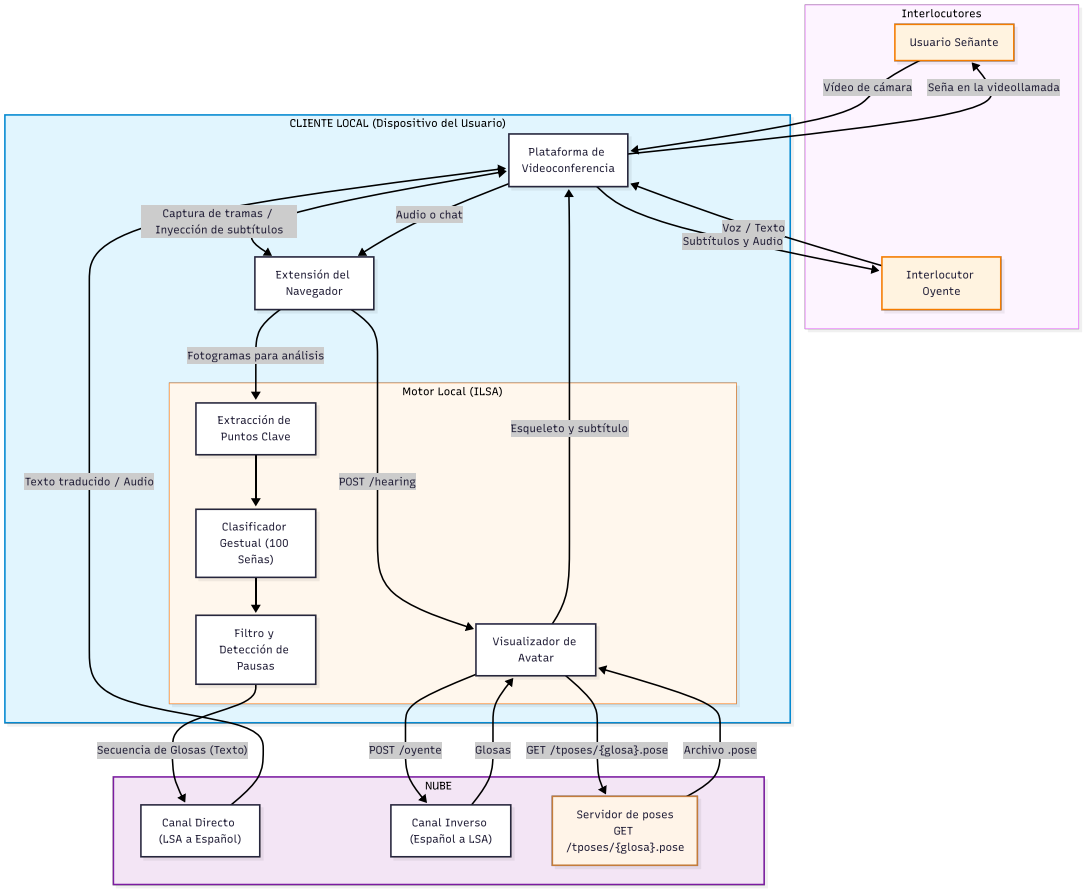


Figura 1.1: Diagrama de bloques de alto nivel del sistema: partición de procesamiento entre el cliente local (captura, extracción de puntos clave, clasificación gestual y motor ILSA) y el servidor semántico remoto en la nube.

El sistema opera a través de dos canales comunicacionales integrados:

- **Canal directo (Señante → Oyente):** Procesa localmente en el cliente las señas aisladas captadas por la cámara web, extrae coordenadas articulares y clasifica las glosas correspondientes. Tras detectar una pausa de inactividad gestual (4 segundos), transmite la secuencia de glosas acumuladas hacia un módulo semántico en la nube que sintetiza oraciones fluidas en español rioplatense, proyecta subtítulos e hipertextos o emite audio sintético hacia el interlocutor oyente.
- **Canal inverso (Oyente → Señante):** Transcribe la voz del oyente, reestructura en la nube la oración en español hacia la sintaxis canónica de la LSA (*Topic-Comment*) y envía la secuencia de glosas resultantes hacia el cliente, donde un avatar tridimensional reproduce las animaciones gestuales o ejecuta deletreo manual dactilológico (carácter por carácter) ante términos fuera del vocabulario base.

1.4.1. Delimitación del vocabulario y asesoramiento lingüístico

El alcance funcional del clasificador gestual está delimitado a un vocabulario objetivo de 100 señas de LSA (cuyo detalle y categorización se presentan en el Apéndice **??**).. Para la selección y estructuración de este corpus se contó con la colaboración y asesoramiento técnico de Miriam Rolls, profesora e intérprete de LSA, quien participó en la delimitación

del recorte léxico. El vocabulario se orientó hacia situaciones de interacción cotidiana y contextos de primera respuesta o emergencia (verbos de auxilio, identificación, parentesco y datos temporales). La validación provista por la especialista garantizó la fidelidad lingüística de las señas seleccionadas para la construcción del corpus, sin que esto constituya una evaluación formal de usabilidad ni un ensayo clínico con usuarios finales en entornos reales de emergencia.

1.4.2. Límites operativos y comportamiento ante entradas no contempladas

El prototipo presenta las siguientes delimitaciones operativas:

- **Delimitación temporal de enunciados:** El cierre de una frase de señas se determina de forma heurística mediante la detección de una pausa física de inactividad gestual de aproximadamente 4 segundos.
- **Rasgos no manuales:** La extracción de características cinemáticas se restringe al esqueleto superior y a las manos, descarta el análisis de expresiones faciales para reducir el consumo de CPU.
- **Entradas con baja confianza o fuera del vocabulario:** En el canal directo, las predicciones del clasificador con baja confianza estadística son descartadas por un filtro anti-rebote. En el canal inverso, las palabras del español que no disponen de una seña equivalente en el léxico de 100 clases son derivadas a la rutina de deletreo dactilológico del avatar; dicho deletreo proyecta los caracteres del término pero no garantiza por sí solo la traducción conceptual del significado si el interlocutor sordo no conoce el vocablo en español oral.
- **Resiliencia ante fallas de red:** En caso de pérdida de conectividad con el servicio semántico remoto, el motor local ILSA conmuta a un modo de contingencia que proyecta las glosas clasificadas en texto directo sin reestructuración sintáctica.

1.4.3. Infraestructura de despliegue

En lo que respecta a la infraestructura de red, el alcance comprende la especificación y diseño formal de una arquitectura elástica y escalable basada en contenedores y balanceadores de carga para un entorno de producción concurrente. Para la evaluación empírica del prototipo desarrollado en este trabajo, la implementación se valida mediante una instancia semántica remota expuesta a través de un túnel seguro.

Capítulo 2

Objetivos

2.1. Objetivo general

Desarrollar un prototipo de sistema de traducción bidireccional entre la LSA y el español rioplatense con procesamiento en tiempo pseudo-real, diseñado para operar sobre plataformas de videoconferencia mediante una arquitectura híbrida cliente-nube, con el fin de mitigar la asimetría en la comunicación expresiva y receptiva, fomentando la autonomía de las personas sordas en la interacción con interlocutores oyentes no señantes.

2.2. Objetivos específicos

- Implementar un módulo local de visión computacional y clasificación gestual basado en la extracción de puntos de referencia corporales para el reconocimiento de un vocabulario de 100 señas de LSA.
- Diseñar e integrar el servicio semántico del canal directo (LSA → Español) para transformar secuencias de glosas aisladas en oraciones fluidas y gramaticalmente correctas en español rioplatense.
- Diseñar e integrar el servicio semántico del canal inverso (Español → LSA) para reestructurar oraciones del español oral a secuencias de glosas alineadas con las reglas morfosintácticas y la estructura canónica *Topic-Comment* de la LSA.
- Implementar un módulo de visualización gestual inversa mediante un avatar capaz de animar las señas resultantes y ejecutar deletreo dactilológico para términos ausentes en el vocabulario.
- Diseñar una arquitectura escalable en la nube para el aprovisionamiento concurrente de los servicios semánticos, lo que desacopla el procesamiento del hardware cliente.
- Resguardar la privacidad de los datos biométricos de los usuarios al ejecutar la extracción de características y la clasificación de forma local en el dispositivo cliente, limitando la transmisión hacia la nube a cargas puramente textuales.
- Desarrollar la arquitectura de software de una extensión para navegadores basados en la tecnología Chromium (tales como Google Chrome, Microsoft Edge, Brave, Opera o Vivaldi) articulada con un motor de ejecución en segundo plano (ILSA).

- Evaluar el desempeño integral del prototipo en términos de exactitud de clasificación con personas no vistas en el entrenamiento, fidelidad semántica bidireccional (métricas BLEU y ROUGE-L), latencia de respuesta y consumo de recursos de hardware, conforme a las métricas y criterios de aceptación definidos en el marco metodológico.

Capítulo 3

Marco teórico, estado del arte y metodología

3.1. Marco teórico

3.1.1. Componentes articulatorios de la LSA

La articulación de la LSA se sostiene sobre la interacción simultánea de dos grandes conjuntos de parámetros lingüísticos:

- **Rasgos manuales:** Comprenden la configuración o forma de la mano, la ubicación o punto de articulación en el espacio corporal, la orientación de la palma y el movimiento o trayectoria cinemática (Massone & Martínez, 2012).
- **Rasgos no manuales:** Incluyen la gesticulación facial (movimiento de cejas, boca y mirada), la inclinación del torso y la postura del cuello (Massone & Martínez, 2012). Estos elementos cumplen un rol determinante para matizar la intensidad de una acción, marcar interrogaciones o delimitar la polaridad en enunciados complejos.

3.1.2. Estructura lingüística de la LSA y traducción *Topic-Comment*

La LSA es una lengua independiente del español oral, dotada de su propia gramática, léxico y reglas sintácticas (Massone & Martínez, 2012). Para abordar informáticamente su traducción, es necesario introducir el concepto de **glosa**: representación escrita en letras mayúsculas que se le asigna a una seña para poder identificarla, transcribirla y analizar su gramática en un formato de texto, sin que esto constituya una traducción literal al español (Massone & Machado, 1993).

A nivel morfosintáctico, la LSA presenta diferencias sustanciales con el español rioplatense:

- **Estructura oracional:** Mientras que el español sigue preferentemente un orden Sujeto-Verbo-Objeto (SVO), la LSA organiza sus proposiciones bajo el principio de *Topic-Comment* (Massone & Martínez, 2012).
- **Ausencia de elementos funcionales:** La LSA carece de artículos determinados e indeterminados («el», «la», «un»), preposiciones gramaticales abstractas («de», «a»,

«con») y verbos copulativos («ser» o «estar») (Massone, 1993; Massone & Martínez, 2012).

- **Marcadores de cierre:** Las partículas interrogativas (`QUÉ`, `DÓNDE`, `CUÁNDO`) y los operadores de negación (`NO`) se posponen obligatoriamente al final de la cláusula oracional.

Bajo el esquema de *Topic-Comment*, el enunciado sitúa en primer lugar el marco temporal y espacial de referencia (el tópico o contexto), seguido por el sujeto, el objeto y finalmente la acción o estado junto con sus modificadores (el comentario):

$$\begin{aligned}\text{Oración LSA} \rightarrow \text{Tiempo} \rightarrow \text{Lugar} \rightarrow \text{Sujeto} \\ \rightarrow \text{Objeto/Sustantivo} \rightarrow \text{Verbo} \rightarrow \text{Negación/Pregunta}\end{aligned} \tag{3.1}$$

Debido a estas diferencias morfosintácticas, la equivalencia entre ambas lenguas no admite un mapeo léxico directo o traducción término a término. Resulta imprescindible incorporar una etapa intermedia de interpretación semántica mediante modelos de lenguaje que traduzca tanto la secuencia de glosas discontinuas provenientes del clasificador a oraciones fluidas en español, como el texto en español a glosas formalmente ordenadas antes de su animación visual.

Delimitación del prototipo y alcance de captura. A pesar de la relevancia lingüística de la gesticulación facial en la LSA, en este trabajo se optó por capturar únicamente los datos de las manos y el esqueleto superior. Se descartó el uso de la malla facial para reducir el consumo de procesamiento en el equipo del usuario y evitar sobrecargar la CPU durante una videollamada.

Esta decisión responde a que las señas seleccionadas para el vocabulario del prototipo son independientes de los gesticulaciones faciales. Aunque algunas señas comparten configuraciones similares entre sí y dependen del contexto para su correcta interpretación, la combinación de pose y manos resulta plenamente suficiente para clasificar el léxico definido sin comprometer la fluidez ni la tasa de fotogramas del sistema.

3.1.3. Visión computacional y reconocimiento basado en esqueletos

El reconocimiento de señas sobre video crudo opera, en cada fotograma, sobre millones de valores de color. Un cuadro de 1920×1080 en RGB contiene

$$1920 \times 1080 \times 3 = 6\,220\,800 \tag{3.2}$$

escalares. Esa representación es sensible a la iluminación, al fondo y a la vestimenta, y su costo impide una inferencia sostenida en la CPU de una computadora que, al mismo tiempo, mantiene una videollamada. El prototipo no clasifica píxeles, clasifica un esqueleto: un conjunto reducido de coordenadas articulares llamadas *landmarks*.

El estimador es MediaPipe Holistic, un marco de código abierto para cadenas de percepción en el dispositivo (Google, 2020; Lugaresi et al., 2019). La solución utilizada, no es un único detector. Es un encadenado de modelos ya entrenados, que corre en el cliente y no transmite la imagen. El primer modelo, BlazePose, localiza a la persona y estima 33 puntos de *pose* (Bazarevsky et al., 2020). Esos puntos cubren un rostro grueso (nariz, ojos, orejas y comisuras), el torso, los brazos y las piernas. A partir de las muñecas de esa *pose*,

el pipeline recorta una región para cada mano. El modelo de manos (*hand*) estima entonces 21 puntos tridimensionales por mano: la muñeca y cuatro articulaciones por dedo (Zhang et al., 2020). En los fotogramas siguientes, la pose previa define la región de búsqueda, y el detector de cuerpo completo no se repite en todos los cuadros. La Figura 3.1 resume ese recorrido.

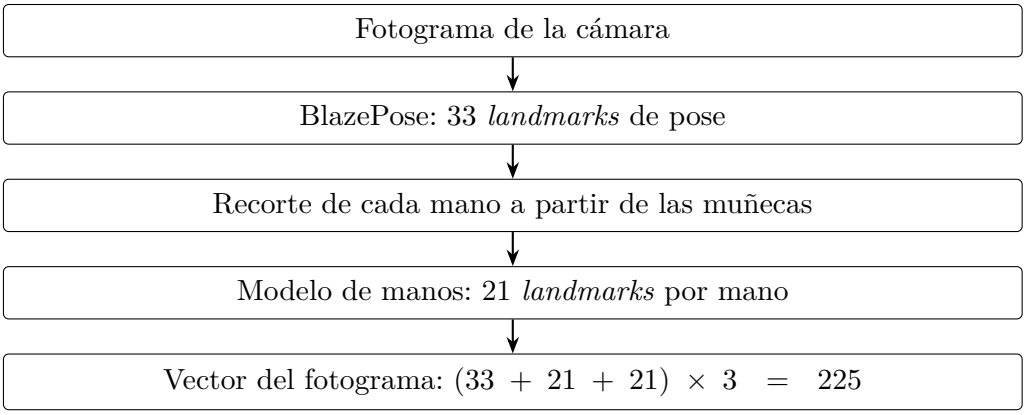


Figura 3.1: Cadena de MediaPipe Holistic retenida en el prototipo. Face Mesh no forma parte del vector de clasificación.

Cada punto aporta una terna (x, y, z). Las componentes x e y se expresan como fracción del ancho y del alto de la imagen, en el intervalo $[0, 1]$. La componente z es una profundidad relativa, en una escala comparable a la de x, y no una distancia en metros hasta la cámara. Con la pose y ambas manos, el descriptor instantáneo queda en

$$(33 + 21 + 21) \times 3 = 225. \tag{3.3}$$

La Tabla 3.1 contrasta esta cifra con el fotograma de color y con la variante que también incluye el rostro denso.

Cuadro 3.1: Escalares por fotograma según la representación de entrada.

Representación	**Puntos**	**Escalares**
Fotograma RGB, 1920×1080	–	6 220 800
Pose y ambas manos	$33 + 42$	225
Pose, manos y Face Mesh	$33 + 42 + 468$	1 629

El paso de 6 220 800 escalares a 225 reduce la entrada en cuatro órdenes de magnitud. Una segmentación píxel a píxel debe reaprender la seña ante cada fondo y cada prenda. El esqueleto entrega a la red la geometría del gesto. El color de la ropa, el tono de la piel y el fondo dejan de ser variables de entrada, y la inferencia queda dentro del presupuesto de una CPU local en tiempo pseudo-real. El modelo completo de malla facial (*Face Mesh*) se excluyó del vector de entrada por razones dimensionales y metodológicas. Por un lado, estimar sus 468 puntos tridimensionales añadiría 1 404 coordenadas adicionales (468×3), un volumen desproporcionado que, frente a un corpus de aproximadamente 100 clips por clase, favorecería que el clasificador memorice la fisonomía del señante antes que la configuración cinemática de la seña. Por otro lado, aunque en la LSA las expresiones de la boca, cejas y movimientos de cabeza delimitan modalidades oracionales (interrogación, negación o

matices gramaticales), incorporar tales rasgos obligaría a filmar variantes expresivas para cada una de las 100 señas lo que sobrecargaria la captura sin aportar a la discriminación del léxico base. En su lugar, el sistema conserva únicamente los puntos faciales provistos nativamente por BlazePose. Si bien estos puntos no reemplazan a *Face Mesh* ni son tratados como rasgos gramaticales por la red, resultan fundamentales para aportar una referencia espacial relativa entre la cara y las manos lo que permite discriminar señas que comparten la misma configuración manual pero difieren en su punto de articulación facial (como el trío `I`, `T` u `OJO`) (Massone & Martínez, 2012).

3.1.4. Arquitectura neuronal temporal: TinySkeletonClassifier

La seña ocupa tiempo. El clasificador recibe una secuencia fija de $T = 16$, cada uno ya reducido a 225 coordenadas, y debe asignar una glosa. Una red que observa los fotogramas por separado pierde el movimiento, por lo que hace falta un modelo de secuencias.

La alternativa recurrente, en particular una LSTM, recorre el tiempo de izquierda a derecha y comprime lo ya visto en un estado oculto. Ese estado puede diluir el inicio del gesto cuando llega la pose final, porque el pasado entra solo a través del paso anterior. El transformador elimina ese cuello de botella (Vaswani et al., 2017). Trata cada fotograma como un elemento de la secuencia y permite que cualquier instante consulte a todos los demás en un solo cálculo.

El mecanismo es el de atención. Para cada fotograma el modelo construye tres vectores: una consulta Q, una clave K y un valor V. La consulta del instante t se compara con la clave de los 16 instantes. Un softmax convierte esas compatibilidades en pesos, y la salida de t es la suma de los valores con esos pesos:

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \tag{3.4}$$

d_k es la dimensión de cada cabeza. Así, el fotograma final puede apoyarse en el inicio del movimiento, en el tramo medio y en la pose sostenida, sin recorrer los cuadros uno por uno. Varias cabezas repiten la cuenta en paralelo. Con 128 dimensiones y 4 cabezas, cada cabeza opera sobre $128/4 = 32$ dimensiones y puede atender a un aspecto distinto del gesto, por ejemplo la trayectoria de la muñeca o la forma de la mano.

Un transformador de gran escala lleva ese mecanismo a secuencias de cientos o miles de elementos y a modelos de 10^7 a 10^8 parámetros. Aquí la secuencia mide 16 fotogramas y el corpus ofrece cerca de 100 clips por clase. En esa escala, más capacidad memoriza los clips de entrenamiento y después pierde exactitud frente a la cámara. El cálculo de la atención, además, crece con el cuadrado del largo: con $T = 16$ los pares son $16^2 = 256$, un costo menor. El límite del problema no es la longitud. Es el dato, y también la CPU del cliente, que debe clasificar mientras sostiene la videollamada.

Por eso el modelo retenido es un transformador pequeño, *TinySkeletonClassifier*, en la línea de las variantes compactas pensadas para recursos limitados (Wu et al., 2020). La configuración es de 128 dimensiones, 4 cabezas y 2 capas, del orden de $4{,}1 \times 10^5$ parámetros. La Tabla 3.2 contrapone esta elección con el transformador de gran escala. La Figura 3.2 y la Tabla 3.3 detallan el camino interno.

Cuadro 3.2: Criterios de adopción de un transformador pequeño frente a uno de gran escala.

Criterio	**Gran escala**	**TinySkeleton**
Longitud de la secuencia	Cientos o miles de elementos	16 fotogramas
Parámetros	10^7–10^8	$\sim 4{,}1 \times 10^5$
Dato de entrenamiento	Corpus masivo	~ 60 clips por clase
Ejecución prevista	GPU o servidor	CPU del cliente

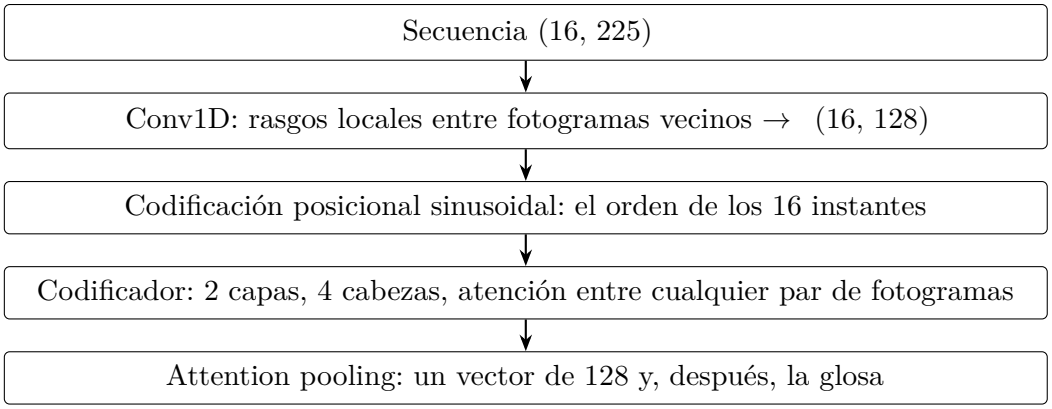


Figura 3.2: Camino de *TinySkeletonClassifier*. La atención actúa sobre 16 fotogramas de 128 dimensiones, no sobre el video.

Cuadro 3.3: Etapas del clasificador y función de cada una.

Etapa	**Forma**	**Función**
Conv1D	$(16, 225) \to (16, 128)$	Lee vecindades cortas y retiene transiciones de dedos y muñeca.
Posición sinusoidal	$(16, 128)$	Incorpora el índice temporal. Sin este vector, el orden de los cuadros no existe para la atención.
Codificador	$(16, 128)$	Relaciona inicio, medio y fin del gesto en dos capas de 4 cabezas.
Attention pooling	$(16, 128) \to (128)$	Asigna un peso a cada fotograma y resume la secuencia. La capa lineal final proyecta el vector a las clases.

La convolución temporal antecede al transformador por una razón de escala. Los cambios de dedos ocurren entre fotogramas contiguos. Una convolución de núcleo corto los lee con pocos parámetros. La atención queda libre para las relaciones largas, que son las que distinguen, por ejemplo, una letra estática de la seña dinámica que reutiliza esa forma de mano.

El cierre no es un promedio uniforme de los 16 estados. Una capa lineal produce un puntaje por fotograma, el softmax lo convierte en un peso y la secuencia se resume como suma ponderada. En el alfabeto y en los dígitos, la pose sostenida aporta más que los cuadros en los que la mano entra o sale del gesto. El promedio uniforme mezcla ambas

cosas. El peso aprendido concentra la decisión en los instantes informativos. El vector resultante, de 128 dimensiones, entra en la capa de clasificación.

3.1.5. Modelos de lenguaje pequeños y adaptación LoRA

Para la traducción semántica bidireccional se emplean SLM de las familias Qwen2.5 (Alibaba Cloud, 2024), SmolLM2 (Allal et al., 2024) y Llama 3.2 (Dubey et al., 2024). Estos modelos se adaptan mediante la técnica LoRA (Hu et al., 2022) la cual congela los pesos base del modelo e inserta matrices de descomposición de bajo rango en las proyecciones de atención y en las capas de entrada y salida (`embed_tokens` y `lm_head`) (Hu et al., 2022; Hugging Face, 2024). Para asegurar su ejecución eficiente en CPU o aceleradores en la nube, los modelos resultantes se cuantizan a 4 bits bajo el formato GGUF mediante `llama.cpp` (Frantar et al., 2022; Gerganov, 2023).

Adaptación LoRA profunda y proyecciones de vocabulario. Para adaptar los modelos de lenguaje sin incurrir en el elevado costo computacional de un reentrenamiento completo (*full fine-tuning*), se aplica la técnica de Adaptación de Bajo Rango (*LoRA*) (Hu et al., 2022). Matemáticamente, la matriz de pesos preentrenados $\mathbf{W}_0 \in \mathbb{R}^{d\times k}$ se mantiene congelada durante todo el proceso, mientras que las actualizaciones de aprendizaje se desacoplan en dos matrices entrenables de menor rango $\mathbf{B} \in \mathbb{R}^{d\times r}$ y $\mathbf{A} \in \mathbb{R}^{r\times k}$ (donde el rango $r \ll \text{mín}(d, k)$) (Hu et al., 2022). De este modo, los pesos resultantes se calculan mediante la combinación lineal:

$$\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r}(\mathbf{BA}) \tag{3.5}$$

donde α es un factor constante de escala que pondera el impacto relativo de las matrices adaptadoras respecto al conocimiento base del modelo (Hu et al., 2022). Desde una perspectiva de arquitectura, esta descomposición reduce en más de un 99 % la cantidad de parámetros optimizables: en lugar de actualizar matrices densas de millones de valores, el modelo retiene su conocimiento preentrenado e instruye únicamente las matrices delgadas $\mathbf{A}$ y $\mathbf{B}$ (Hu et al., 2022).

Además de inyectar adaptadores en las capas de atención convencionales (`q_proj`, `k_proj`, `v_proj`, `o_proj`), la arquitectura del traductor incorporó de forma deliberada adaptadores LoRA sobre las capas límite de entrada y salida:

- **`embed_tokens`:** Adapta la proyección inicial para transformar la secuencia de 100 glosas en mayúsculas en representaciones vectoriales densas que capturan la sintaxis propia de la LSA (Hu et al., 2022).
- **`lm_head`:** Ajusta la capa lineal de desproyección final lo que permite a la red asignar probabilidades precisas sobre los artículos, preposiciones y flexiones verbales requeridas en español rioplatense (Hu et al., 2022).

Mecánica de cuantización Q4_K_M A partir del binario consolidado en precisión de 16 bits, se aplica el algoritmo de cuantización por bloques `Q4_K_M` provisto por `llama.cpp` (Frantar et al., 2022; Gerganov, 2023). A diferencia de los métodos uniformes tradicionales donde todos los parámetros se reducen a la misma precisión, la estrategia de k-cuants (*k-quants*) adopta un esquema heterogéneo basado en la sensibilidad de cada estructura neuronal:

- **Tensores de auto-atención y activación (`q_proj`, `v_proj`, `gate_proj`):** Se comprimen a una representación de 4 bits por parámetro, organizados en superbloques que almacenan escalas y mínimos locales, lo que permite reconstruir la distribución de pesos con mínima pérdida de dispersión.
- **Capas críticas de alimentación hacia adelante y normalización:** Se retienen en una precisión mixta de 6 bits (*medium variant*) para proteger las capas más propensas a generar desviaciones sintácticas o degradación en métricas de generación.

3.1.6. Animación y representación visual mediante Avatar

El canal inverso exige, para cada glosa, una forma visual que conserve la configuración de la mano, el punto de articulación y la trayectoria. En la síntesis de lenguas de señas conviven tres familias de representación.

La primera es el avatar paramétrico. El personaje posee una malla, un esqueleto de huesos de longitud fija y, en el rostro, *blendshapes*: pesos que mezclan expresiones sobre la misma geometría. La animación interpola rotaciones de esos huesos o esos pesos. Conducir el muñeco desde una persona filmada exige un *retarget*, esto es, el paso de posiciones de *landmarks* a rotaciones de otro cuerpo (**cabello2010threejs**). Origen y destino difieren en representación. MediaPipe Holistic entrega coordenadas en el plano de la imagen y una profundidad de la mano relativa a la muñeca, en una escala distinta de una distancia en metros. El esqueleto del avatar exige rotaciones sobre sus propias longitudes. Si esas longitudes se respetan, las yemas quedan fuera de los puntos medidos. Si una cinemática inversa alarga el miembro hasta el punto, la mano pierde su configuración. En el alfabeto manual esa diferencia cambia la letra. Los estimadores pensados para avatares de cámara, como Kalidokit, resumen cada dedo en un único ángulo de rizo y confunden configuraciones próximas. Los personajes tridimensionales de Hand Talk y de HETAH pertenecen a esta familia (Hand Talk, 2025; HETAH, 2024).

La segunda familia reproduce un diccionario medido. Cada glosa corresponde a un clip de *landmarks* estimado sobre un video de referencia, archivado por ejemplo en formato `.pose` (**moryossef2021poseformat**). El enunciado es la concatenación de esos clips: el sistema vuelve a dibujar una seña ya registrada. El Señario de la CAS y aplicaciones de consulta como LSApp operan sobre esta lógica léxica. La fonología de las lenguas de señas separa el núcleo significativo de la preparación y de la retracción (**liddell1989phonological**). Entre el final de un clip y el inicio del siguiente, una interpolación breve de los puntos cubre el salto visual. Ese puente es un recurso gráfico. La coarticulación pertenece a la lengua: el final de una seña anticipa la siguiente, y un puente cartesiano deja ese fenómeno fuera de su alcance.

La tercera familia genera el movimiento a partir del texto, sin un clip léxico, mediante un cuerpo paramétrico o un modelo de difusión. La seña resulta entonces una muestra del modelo.

La Tabla 3.4 contrapone las dos familias cuya fuente es un video ya señado. Un término ausente del léxico, como un nombre propio o un tecnicismo, se resuelve en las lenguas de señas por deletreo manual: la palabra se articula letra por letra con el alfabeto dactilológico. El deletreo es productivo; el diccionario, cerrado. La representación adoptada en este trabajo, y el controlador que la ejecuta, se describen en la Sección 5.3.

Cuadro 3.4: Familias de representación visual de una glosa.

Criterio	**Avatar paramétrico**	**Diccionario de poses**
Unidad de animación	Rotación de hueso o peso de *blendshape*.	Trayectoria de *landmarks*.
Cuerpo	Esqueleto de longitudes fijas.	Proporciones de la persona filmada.
Glosa fuera del léxico	Animación nueva o *retarget*.	Deletreo sobre clips del alfabeto.
Límite propio	La configuración manual puede quedar fuera del esqueleto.	Un punto perdido arrastra la interpolación.

3.1.7. Arquitectura híbrida de partición Cliente-Nube

El concepto de **tiempo pseudo-real** designa a aquellos sistemas de procesamiento interactivo donde la respuesta computacional no ocurre de forma estrictamente instantánea ni en el orden de los microsegundos, pero cuyos retardos temporales permanecen dentro de un umbral operativo tolerable para la interacción humana. A diferencia de los sistemas de tiempo real estricto (*hard real-time*), el tiempo pseudo-real admite una latencia acotada resultante de la segmentación temporal por pausas de inactividad, el empaquetado de datos y el tiempo de inferencia de los modelos semánticos en la nube, sosteniendo la fluidez del diálogo bilingüe en la videollamada.

3.1.8. Captura de medios en la Web y aislamiento en extensiones (Manifest V3)

La transmisión y el procesamiento de video en tiempo pseudo-real dentro del navegador web se fundamentan en las especificaciones del estándar W3C Media Capture (W3C, 2024). La interfaz nativa `navigator.mediaDevices.getUserMedia()` permite solicitar al sistema operativo el acceso al flujo de la cámara física. No obstante, los motores de los navegadores modernos gestionan el hardware de captura bajo un esquema de acceso exclusivo: cuando una aplicación web (como la plataforma Google Meet) toma el control del dispositivo de video, el hardware queda bloqueado y no admite el consumo concurrente por parte de otros procesos sin generar conflictos de permisos o interrupciones en la transmisión.

Para resolver esta restricción sin alterar el flujo nativo de la videoconferencia, el estándar W3C define el método `canvas.captureStream()`, el cual permite sintetizar un objeto `MediaStream` dinámico a partir del renderizado fotograma a fotograma sobre un lienzo virtual (`<canvas>`). Este flujo sintético sustituye de manera transparente la entrada de cámara nativa y permite la inyección de subtítulos y el procesamiento cinemático en paralelo.

Por otro lado, la arquitectura de extensiones en navegadores Chromium bajo la especificación Manifest V3 (Google Chrome Developers, 2024) impone restricciones de seguridad estrictas mediante su Política de Seguridad de Contenido, las cuales prohíben la ejecución de código dinámico y restringen el acceso al DOM en scripts de fondo (*service workers*). Para ejecutar modelos de visión computacional basados en WebAssembly e hilos auxiliares (*workers*) sin congelar la pestaña principal de la videollamada, la plataforma requiere un esquema de desacoplamiento en dos entornos aislados:

- **Documento en segundo plano (*offscreen*):** Entorno sin interfaz gráfica habilitado por la API `chrome.offscreen` para gestionar la recepción de fotogramas comprimidos y coordinar la mensajería asíncrona entre la página y el backend.
- **Entorno de ejecución aislado (*sandbox*):** Marco con restricciones de red pero con permisos para instanciar entornos de ejecución de WebAssembly y ejecutar hilos paralelos de inferencia en segundo plano.

3.2. Estado del arte

3.2.1. Desarrollos en Argentina y la región

En el ámbito nacional, las iniciativas tecnológicas orientadas a la comunidad sorda han estado enfocadas principalmente en la enseñanza y la consulta de vocabulario. El Diccionario de Señas del INJU opera como un índice léxico en línea sin capacidades de reconocimiento dinámico (INJU, 2024). La Confederación Argentina de Sordos (CAS) promueve herramientas pedagógicas bilingües de carácter estático (Confederación Argentina de Sordos, 2024). En cuanto a la animación digital, la aplicación móvil LSApp (Barán, 2018) introdujo un personaje tridimensional capaz de reproducir entradas del diccionario y deletreo manual, por lo que funciona como material de consulta unilateral sin integración a videoconferencias.

En el plano asistencial comercial, la plataforma Blessit (Blessit, 2024) y servicios públicos como la Línea 144 (Ministerio de Justicia de la Nación, 2024) ofrecen teleinterpretación mediante videollamada con operadores humanos. Aunque efectivos, estos esquemas están supeditados a la disponibilidad de personal, generan demoras de conexión y comprometen la intimidad de los interlocutores en intercambios de índole privada.

Académicamente, el corpus LSA64 (Ronchetti et al., 2016) constituye el principal antecedente en reconocimiento automático local; no obstante, su captura controlada mediante guantes de colores restringe su aplicación directa sobre cámaras web comerciales sin aditamentos físicos, limitación observada en desarrollos posteriores con redes recurrentes (Mindlin, 2021).

3.2.2. Sistemas internacionales de reconocimiento y síntesis

A nivel internacional, el referente en reconocimiento unidireccional es SignAll Technologies (SignAll Technologies, 2026), enfocado en la traducción de ASL hacia inglés escrito mediante visión computacional y modelos lingüísticos. Sin embargo, su enfoque prioriza que el oyente comprenda a la persona sorda y omite la devolución gestual accesible. Por su parte, corpus a gran escala como WLASL (Li et al., 2020) han impulsado el reconocimiento léxico en video de ASL, aunque sin foco en la traducción oracional bidireccional ni en la LSA. En la dirección inversa (texto a señas), la empresa Hand Talk (Hand Talk, 2025) lidera la región con avatares 3D (Hugo y Maya) que traducen texto escrito a Libras y ASL. De igual forma, el avatar Iris/Juan desarrollado por la fundación HETAH (HETAH, 2024) en Colombia realiza síntesis gestual a partir de oraciones en español. No obstante, ninguna de estas plataformas comerciales unifica reconocimiento continuo de LSA, traducción semántica contextual y síntesis gestual interactiva dentro del flujo nativo de una videollamada bajo un esquema híbrido y escalable.

3.3. Metodología de desarrollo

El proyecto se rigió bajo un modelo de ciclo de vida iterativo e incremental (Pressman & Maxim, 2014), estructurado en 6 fases principales:

1. **Curación de datos y formalización léxica:** delimitación de 100 señas de LSA junto a Miriam Rolls; diseño del set de grabación; captura de clips en video 1080p (16:9); y generación del dataset de coordenadas vectoriales normalizadas.

2. **Entrenamiento y diagnóstico del clasificador gestual:** experimentación de arquitecturas temporales (pruebas desde 8 a 32 frames); optimización de hiperparámetros mediante marcos automatizados (Akiba et al., 2019); desarrollo de instrumentación de diagnóstico de confusiones mediante análisis AUC de Mann-Whitney.

3. **Desarrollo de los módulos semánticos bidireccionales y diseño de despliegue:** ingeniería de prompts; fine-tuning con LoRA sobre modelos Qwen2.5 (Alibaba Cloud, 2024), SmolLM2 (Allal et al., 2024) y Llama 3.2 (Dubey et al., 2024) (traducción directa e inversa *Topic-Comment* (Massone & Martínez, 2012)); cuantización a binarios GGUF (Gerganov, 2023); evaluación de fidelidad sintáctica (BLEU, ROUGE-L, METEOR); diseño conceptual de la arquitectura elástica en la nube; y prueba del servicio semántico con Cloud Run.

4. **Integración y renderizado gestual:** acoplamiento del pipeline en el motor local ILSA mediante patrones de diseño desacoplados (Gamma et al., 1994); comunicación con el servidor semántico desacoplado; e intercepción de video en Google Meet mediante Canvas sintético y extensión Manifest V3 (Google Chrome Developers, 2024; W3C, 2024).

5. **Avatar e integración visual inversa:** desarrollo del controlador del avatar para la representación esquelética de las glosas del canal inverso; implementación del subsistema de contingencia para deletreo manual dactilológico ante términos ausentes en el léxico; integracion en la extension.

6. **Fase de pruebas y evaluación de campo:** pruebas integrales de latencia, usabilidad y robustez frente a escenarios reales de videoconferencia.

3.3.1. Declaración sobre el uso de herramientas de inteligencia artificial

Herramientas empleadas. Durante las fases de diseño, implementación y redacción se utilizaron exclusivamente dos herramientas:

- **Cursor:** Entorno de desarrollo integrado (IDE) asistido por inteligencia artificial, empleado para la navegación contextual del repositorio, autocompletado sintáctico y asistencia en la edición de scripts en Python y JavaScript.

- **Gemini (Google):** Modelo de lenguaje utilizado como soporte interactivo de consulta técnica, facilitador en la asimilación de conceptos teóricos y asistencia en la redacción del informe.

Alcance y propósito de su utilización La intervención de dichas herramientas se limitó a:

- **Aprendizaje guiado y asimilación conceptual:** Utilización de Gemini como interlocutor de estudio y consulta técnica para comprender en profundidad conceptos que excedían la currícula tradicional de grado, tales como las complejidades de la cuantización GGUF Q4_K_M con `llama.cpp` (Gerganov, 2023), los fundamentos de adaptación de bajo rango (LoRA) en capas densas de modelos autorregresivos (Hu et al., 2022), y los esquemas de comunicación asíncrona entre contextos aislados (*offscreen* y *sandbox*) exigidos por Chrome Manifest V3 (Google Chrome Developers, 2024).
- **Asistencia en el desarrollo de software:** Sugerencias de refactorización de código, resolución de incompatibilidades de sintaxis en el entorno Chrome Extensions Manifest V3 (aislamiento de páginas *offscreen* y *sandbox*) (Google Chrome Developers, 2024) y resolución de dependencias concurrentes en Python.
- **Consultas sobre APIs y formatos:** Apoyo en la parametrización de llamadas a MediaPipe Holistic (Google, 2020) y en la sintaxis de cuantización a formato GGUF mediante herramientas de línea de comandos (Gerganov, 2023).
- **Avatar:** Consulta sobre la diferencia entre un esqueleto medido y el *retarget* hacia huesos VRM, el archivo `.pose` (**moryossef2021poseformat**) y el empalme de clips.
- **Edición y formato del documento:** Corrección resolución de errores en la redacción del informe.

Límites y responsabilidad de autoría. Se deja constancia de que ninguna de las herramientas generativas reemplazó la labor intelectual, analítica ni experimental de los autores. La concepción de la arquitectura híbrida, la recolección y curación del corpus de video de 100 señas validado por Miriam Rolls, el diseño e iteración de las redes neuronales (TinySkeletonClassifier), la ingeniería de prompts y fine-tuning de los modelos de lenguaje, el diagnóstico de captura y el análisis de resultados fueron ejecutados y validados íntegramente por los estudiantes. Los autores asumen la total responsabilidad técnica y académica por el contenido del presente informe y del código fuente asociado.

3.4. Herramientas y tecnologías empleadas

Cuadro 3.5: Conjunto de tecnologías del sistema integrado

Categoría	**Herramienta**	**Rol en el sistema**
Entorno y Lenguajes	Python 3.11, JavaScript	Lógica de backend, motor local, servidor semántico y extensión web.
Visión Computacional	MediaPipe Holistic (Google, 2020), OpenCV	Extracción de coordenadas articulares corporales y manipulación de video.
Aprendizaje Profundo	PyTorch 2.6	Inferencia local del clasificador gestual TinySkeletonClassifier.
Modelos de Lenguaje	Transformers (Hugging Face, 2024), PEFT, Unsloth	Fine-tuning eficiente con adaptadores LoRA (Hu et al., 2022) para traducción directa e inversa.
Inferencia de Lenguaje	`llama.cpp` (Gerganov, 2023), GGUF	Servidor de inferencia semántica optimizado en CPU/GPU remota.
Comunicación y Backend	FastAPI (Ramírez, 2018), Uvicorn	Servidor de interfaz local (ILSA) y microservicio semántico desacoplado.
Servicio en la nube	Google Cloud Run	Instancia única del servicio semántico usada en las pruebas. El clúster elástico de producción no forma parte de esa instancia.
Integración Web	Chrome Extensions (MV3) (Google Chrome Developers, 2024)	Intercepción de media mediante Canvas `captureStream()` en Google Meet (W3C, 2024).
Síntesis de Salida	pyttsx3, Avatar	Emisión de voz local hacia el oyente y animación visual gestual hacia el señante.

Capítulo 4

Análisis de requerimientos

4.1. Requerimientos funcionales

- **RF-01 — Captura y extracción local de landmarks:** Capturar el flujo de video del usuario señante en el cliente y extraer las coordenadas articulares tridimensionales normalizadas de torso y manos en tiempo real mediante visión computacional (Google, 2020).
- **RF-02 — Clasificación gestual temporal local:** Clasificar localmente secuencias fijas de coordenadas en una de las 100 clases de LSA soportadas y utilizar arquitecturas de aprendizaje profundo para reportar la glosa asignada y su confianza.
- **RF-03 — Traducción semántica directa remota (LSA → Español):** Transmitir las glosas detectadas hacia el servicio semántico en la nube y recibir una oración gramaticalmente correcta en español rioplatense generada por un modelo de lenguaje adaptado
- **RF-04 — Emisión hacia el oyente:** Presentar la traducción en texto superpuesto en pantalla y/o sintetizarla como audio oral para el interlocutor oyente de forma concurrente.
- **RF-05 — Captura del turno del interlocutor oyente:** Capturar la voz o texto del oyente en la videollamada durante su turno de intervención con el fin de ser procesada para su uso en el canal de retorno bidireccional.
- **RF-06 — Traducción semántica inversa remota (Español → LSA):** Transmitir el texto en español emitido por el oyente hacia el servicio semántico en la nube y obtener una secuencia ordenada de glosas bajo la estructura gramatical *Topic-Comment* canónica de la LSA.
- **RF-07 — Representación visual mediante Avatar LSA:** Recibir las glosas generadas por el canal inverso y renderizar localmente en pantalla la animación tridimensional de la seña correspondiente, y ejecutar deletreo manual dactilológico en términos fuera del léxico.
- **RF-08 — Integración con Google Meet:** Inyectar el video procesado y los canales de comunicación dentro de la pestaña de videollamada.

4.2. Requerimientos no funcionales

- **RNF-01 — Latencia y tiempo de respuesta:** La delimitación de cierre de enunciado debe operar tras un umbral de 4 segundos sin actividad gestual. La inferencia semántica remota y retorno al cliente no debe superar los 1,5 segundos bajo conexiones de banda ancha estándar, apoyándose en la optimización de pesos cuantizados.
- **RNF-02 — Privacidad de imagen y video:** Ningún flujo de video crudo o imagen de la cámara web debe ser transmitido hacia servidores externos o servicios en la nube; el intercambio de red con el backend se restringe exclusivamente a cargas textuales livianas (glosas o texto transcripto).
- **RNF-03 — Robustez en reconocimiento en vivo:** El clasificador gestual local debe mantener un rendimiento superior al 75 % en condiciones de iluminación doméstica estándar.
- **RNF-04 — Desacoplamiento y escalabilidad del servicio semántico:** La arquitectura del backend semántico debe estar concebida para operar de forma desacoplada y elástica, lo que permita su despliegue en infraestructuras con auto-escalado horizontal a través de interfaces REST tipadas.
- **RNF-05 — Usabilidad y no invasividad:** La interfaz sobre la videollamada debe ofrecer indicadores visuales de estado que no ocluyan la visibilidad del interlocutor para mitigar la brecha comunicacional receptiva y expresiva.

4.3. Restricciones

- **RC-01 — Alcance léxico:** El vocabulario reconocido por el clasificador está acotado a 100 señas validadas por profesionales de LSA, enmarcadas en las pautas pedagógicas y lingüísticas vigentes.
- **RC-02 — Despliegue en la nube del prototipo:** Las pruebas no despliegan la arquitectura elástica del servicio semántico. La validación usa Cloud Run, de Google Cloud.
- **RC-03 — Compatibilidad de plataforma cliente:** El empaquetado del motor local (ILSA) y la extension estan diseñados para el sistema operativo Windows y navegadores basados en Chromium bajo las especificaciones de seguridad vigentes.
- **RC-04 — Dependencia de visión:** El clasificador depende del correcto funcionamiento de MediaPipe Holistic para la extraccion de landmarks.
- **RC-05 — Recursos del cliente objetivo:** El componente local del prototipo debe ser capaz de operar en laptops estándar (procesadores de 4 núcleos y 8 GB de memoria RAM, sin placa gráfica dedicada).

4.3.1. Especificación del entorno de ejecución cliente

A fin de asegurar la accesibilidad y democratización del sistema, se definió un hardware objetivo representativo del equipamiento informático estándar presente en ámbitos educativos y laborales:

Cuadro 4.1: Requisitos de hardware y software del dispositivo cliente

Componente	**Requisitos mínimos (Dispositivo base)**	**Requisitos recomendados**
Procesador (CPU)	Intel Core i3 (10ª gen.) / AMD Ryzen 3 (serie 3000) (4 núcleos / 4 hilos, base $\geq$ 2,0 GHz, soporte AVX2).	Intel Core i5 (11ª gen.+) / AMD Ryzen 5 (serie 4000+) (6 núcleos / 12 hilos).
Memoria RAM	8 GB DDR4 ($\geq$ 3,0 GB libres para el entorno de ejecución).	16 GB DDR4 / DDR5.
Gráficos (GPU)	Gráficos integrados (Intel UHD Graphics 620 / AMD Radeon Vega 3) con soporte WebGL.	Intel Iris Xe / AMD Radeon 600M+ o GPU dedicada básica.
Almacenamiento	Disco de estado sólido (SSD) con 2 GB de espacio libre.	SSD NVMe con $\geq$ 5 GB libres.
Cámara web	Resolución 720p (1280×720) a 24/30 FPS.	Resolución 1080p a 30 FPS.
Conectividad	Conexión de banda ancha ($\geq$ 10 Mbps bajada / $\geq$ 3 Mbps subida, latencia $<$ 80 ms).	Conexión por fibra óptica ($\geq$ 50 Mbps).
Entorno de software	Windows 10/11 (64 bits), Google Chrome 115+ (soporte Manifest V3).	Windows 11 (64 bits), Google Chrome última versión estable.

Criterio técnico de selección del procesador base La especificación de la CPU base no fue adoptada de manera arbitraria, sino deducida a partir de la descomposición de hilos requerida en ejecución simultánea:

1. **Carga WebRTC en navegador:** Google Meet demanda aproximadamente entre 1 y 1,5 núcleos para la decodificación y codificación de video en tiempo real.

2. **Extracción (MediaPipe Holistic):** La ejecución sobre WebAssembly en el contexto aislado del navegador insume la capacidad de 1 núcleo dedicado para mantener una cadencia de 24 a 30 FPS.

3. **Inferencia gestual e interprocesos (Motor ILSA):** El servidor local FastAPI y el modelo neuronal en CPU (*TinySkeletonClassifier*) requieren disponibilidad de cómputo vectorial (instrucciones AVX2/FMA) para garantizar latencias de clasificación inferiores a los 2 ms sin contienda de recursos.

4.4. Casos de uso

4.4.1. Casos de uso esenciales en formato breve

- **CU-01 — Conversar con otra persona (Modo Sordo):** El usuario sordo señа frente a la cámara web durante la videollamada; el sistema captura y clasifica localmente las señas, solicita la traducción remota y proyecta el mensaje como texto o voz para el oyente.
- **CU-02 — Configurar mano dominante:** El usuario señante selecciona si su mano hábil es la derecha o la izquierda.
- **CU-03 — Configurar canal de salida:** El usuario señante selecciona si prefiere que la interpretación sea en formato de subtítulos, voz sintetizada en off o ambas alternativas simultáneamente.
- **CU-04 — Traducir español a LSA mediante Avatar (Modo Oyente):** El usuario oyente habla o escribe durante la llamada; el sistema solicita la estructuración remota en glosas LSA y el avatar reproduce localmente las señas o las deletrea visualmente.

4.4.2. Casos de uso expandidos

Identificador	**CU-01: Conversar con otra persona (Modo Sordo)**		
Actor principal	Usuario señante		
Precondiciones	El usuario debe estar unido a una llamada.		
Postcondiciones	El usuario no señante es capaz de leer/escuchar lo que el usuario señante le dice		
Escenario Principal de éxito (flujo básico)		**Usuario señante**	**Sistema**
	1	Realiza las señas correspondientes a lo que quiere comunicar	
	2		El Sistema procesa la seña, extrae los landmarks, clasifica la seña y la carga en un buffer temporal.
	3	Si necesita realiar otra seña, vuelve al paso 1. Sino espera 4 segundos.	
	4		El Sistema envia el buffer temporal al módulo semántico, que se encarga de interpretar y mostrar por pantalla el resultado
Extensiones (flujos alternativos)	-		
Requisitos especiales	-		

<table>
<tr><td>Identificador</td><td colspan="3">CU-02: Configurar mano dominante</td></tr>
<tr><td>Actor principal</td><td colspan="3">Usuario señante</td></tr>
<tr><td>Precondiciones</td><td colspan="3">El usuario debe estar unido a una llamada.</td></tr>
<tr><td>Postcondiciones</td><td colspan="3">Queda configurada la mano hábil para el módulo clasificador</td></tr>
<tr><td rowspan="4">Escenario Principal de éxito (flujo básico)</td><td></td><td>Usuario señante</td><td>Sistema</td></tr>
<tr><td>1</td><td></td><td>Muestra dos botones, "Seleccionar mano habil:" "izquierda" o "derecha"</td></tr>
<tr><td>2</td><td>Hace click en el botón correspondiente</td><td></td></tr>
<tr><td>3</td><td></td><td>Configura que la extracción y tratamiento de landmarks sea con ese tipo de mano habil</td></tr>
<tr><td>Extensiones (flujos alternativos)</td><td colspan="3">-</td></tr>
<tr><td>Requisitos especiales</td><td colspan="3">-</td></tr>
</table>

Identificador	**CU-03: Configurar canal de salida del oyente**		
Actor principal	Usuario señante		
Precondiciones	El usuario debe estar unido a una llamada.		
Postcondiciones	Queda configurada la salida del módulo semantico		
Escenario Principal de éxito (flujo básico)		**Usuario señante**	**Sistema**
	1	Hace click en el botón "configuración de interpretación"	
	2		Muestra 3 opciones, "Texto" "Voz en off" o "Ambas"
	3	Hace click en la opción de preferencia	
	4		Configura la salida con la preferencia del usuario
Extensiones (flujos alternativos)	-		
Requisitos especiales	-		

Identificador	**CU-04: Traducir español a LSA mediante Avatar (Modo Oyente)**		
Actor principal	Usuario no señante		
Precondiciones	El usuario debe estar unido a una llamada. Se debe haber realizado una interpretación		
Postcondiciones	-		
Escenario Principal de éxito (flujo básico)		**Usuario señante**	**Sistema**
	1	Habla o ingresa un mensaje de texto al chat	
	2		Captura el flujo de audio y lo transcribe a texto en español
	3		Procesa el mensaje con el módulo semántico y obtiene una secuencia de glosas
	4		Renderiza la animación de las glosas sobre la cámara del usuario
Extensiones (flujos alternativos)	-		
Requisitos especiales	-		

Capítulo 5

Diseño del Sistema

5.1. Arquitectura del sistema

El diseño adopta una arquitectura híbrida. La extracción de puntos y la clasificación gestual corren en el equipo del usuario. El módulo semántico corre como servicio remoto.

5.1.1. Justificación de la partición y modelo de privacidad

Se probó una primera configuración con las tres etapas en el cliente: extracción, clasificación y traducción semántica. La extracción y la clasificación conviven con la videollamada. La demanda de memoria RAM del modelo de lenguaje obliga a retirarlo del equipo del usuario y a publicarlo como servicio en la nube. Se utiliza un filtro para evitar repeticiones ilógicas, y hacia el servicio sale solo el texto del turno en curso, ya cerrado, en forma de glosas o de oración. El video y los puntos articulares permanecen en el equipo.

5.1.2. Flujo integral de datos entre componentes.

La dinámica operativa del sistema conecta el navegador, el motor local y dos servicios remotos. La Figura 5.1 fija esa partición. El video y los puntos articulares quedan en el equipo. Hacia la nube sale texto: glosas en el canal directo, u oración en el canal inverso. El servicio de poses no traduce. Devuelve el archivo de la glosa.

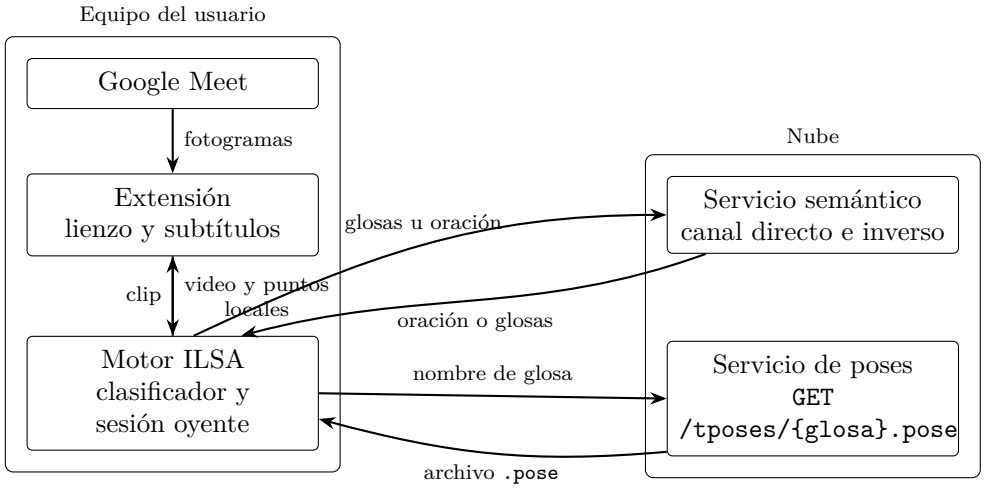


Figura 5.1: Diagrama de componentes. El servicio de poses queda aparte del traductor y entrega el archivo de seña.

Para modelar la interacción temporal y la secuencia de mensajes se analizan a continuación los dos canales comunicacionales de manera independiente.

Dinámica temporal del Canal Señante → Oyente (Modo Sordo)

En la Figura 5.2 se modela el ciclo de vida completo de un enunciado en LSA, desde la articulación gestual frente a la cámara web hasta la emisión de subtítulos y voz sintética hacia el interlocutor oyente.

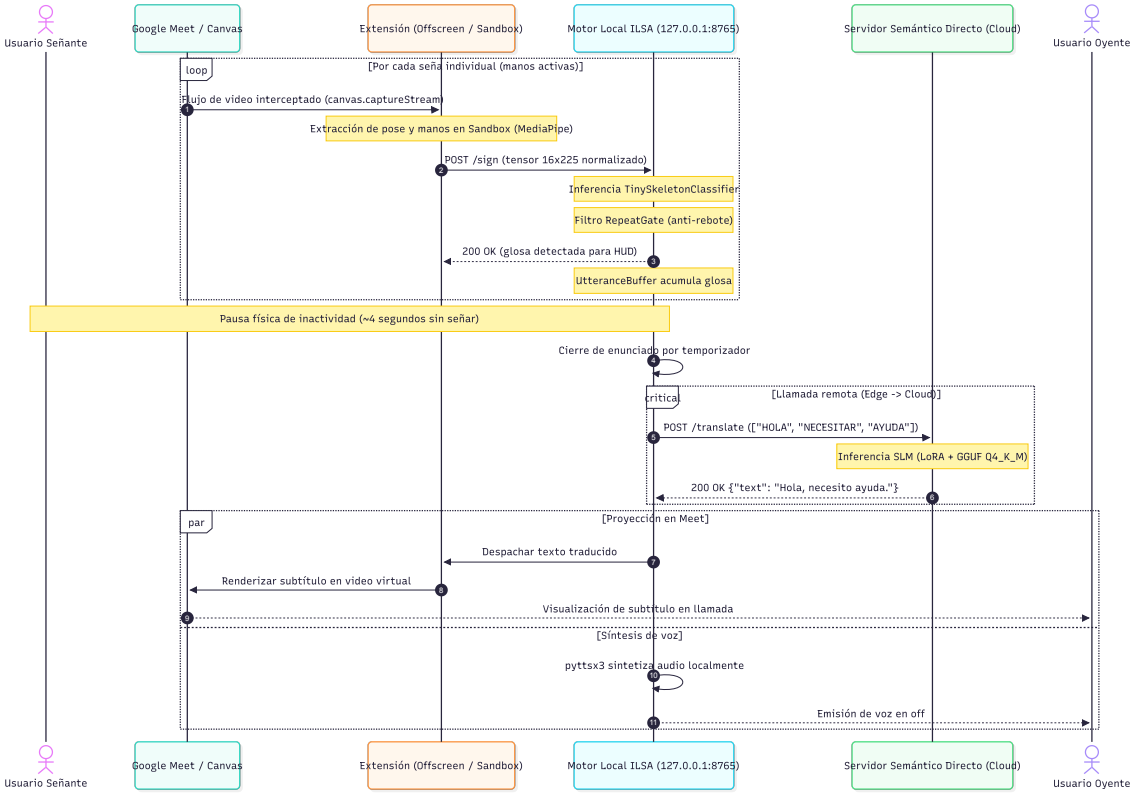


Figura 5.2: Diagrama de secuencia del Canal Señante → Oyente: bucle continuo de clasificación de señas en el borde, delimitación de pausas e invocación al servidor semántico remoto.

Dinámica temporal del Canal Oyente → Señante (Modo Oyente)

En la Figura 5.3 se describe la secuencia temporal que transforma el mensaje oral o escrito del oyente en representaciones visuales comprensibles en LSA para el usuario sordo.

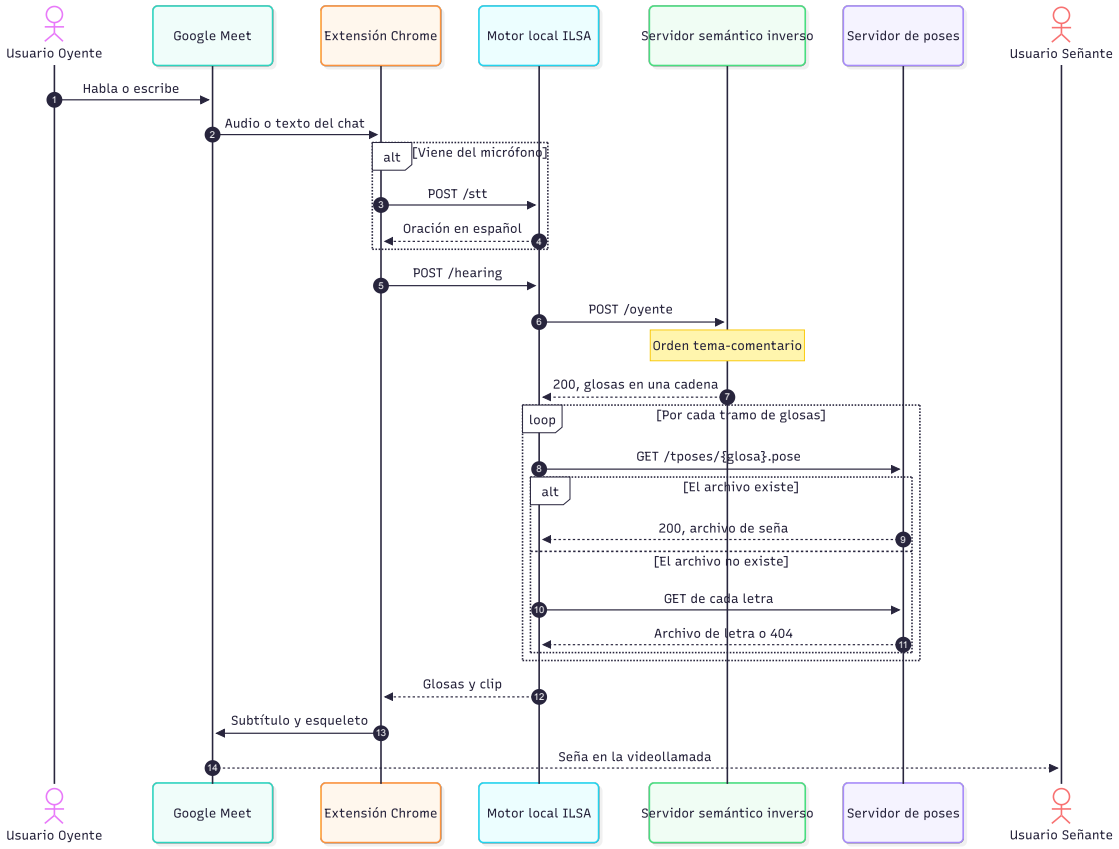


Figura 5.3: Diagrama de secuencia del Canal Oyente → Señante: transcripción del habla o mensaje, reestructuración gramatical Topic-Comment en la nube y animación o deletreo dactilológico en el cliente.

La Figura 5.4 baja al interior de ese canal. ILSA pide el archivo, y arma la lista de fotogramas. El lienzo no recalcula el puente: elige el fotograma según el tiempo transcurrido. Si el archivo no existe, ILSA pide las letras al mismo servicio.

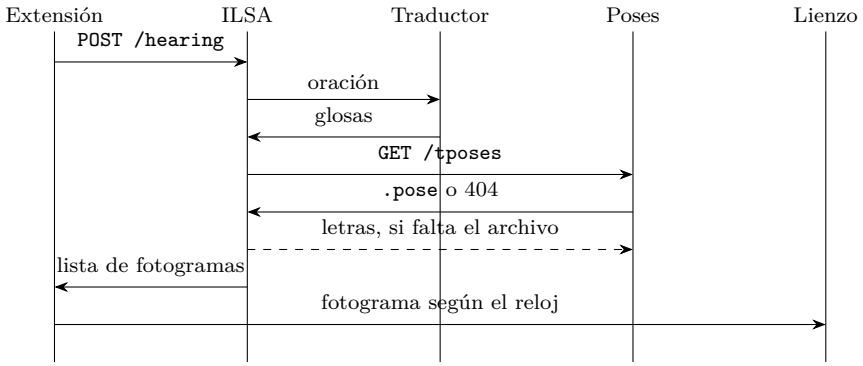


Figura 5.4: Secuencia de componentes del canal oyente. El servicio de poses responde el archivo o un 404.

5.2. Diseño de contratos de API REST

La comunicación entre el cliente y los servicios se rige por contratos tipados en formato JSON.

5.2.1. API local del motor ILSA

El motor ILSA expone una interfaz HTTP de alcance local. El contrato queda restringido al bucle de retorno del equipo anfitrión: la extensión de Meet actúa como cliente y el proceso del motor actúa como servidor. El diseño cubre diagnóstico, consulta de estado, apertura de sesión gestual, ingesta de una seña recortada, aviso de actividad y cierre de enunciado. El cuerpo de las peticiones y respuestas usa JSON.

Los métodos admitidos son `GET` y `POST`. Una respuesta correcta usa el código `200 OK`. Un cuerpo JSON inválido o incompleto produce `400 Bad Request`. Si el motor aún no posee una sesión activa, las rutas de operación responden `503 Service Unavailable`.

Cuadro resumen de endpoints

El Cuadro 5.1 reúne las rutas del contrato de interpretación gestual (modo sordo). Cada ruta es independiente en el método HTTP; el estado conversacional de glosas reside en el búfer de enunciado del motor, no en el cliente.

Cuadro 5.1: Endpoints de la API local del motor ILSA.

Método	**Ruta**	**Contrato**	**Códigos**
GET	`/health`	Diagnóstico de disponibilidad del clasificador local y de la conexión con el servicio semántico remoto.	200
GET	`/state`	Consulta de las glosas acumuladas en el búfer temporal y del último texto traducido.	200, 503
POST	`/session`	Inicio de sesión gestual y fijación de lateralidad mediante `{"left_handed": bool}`.	200, 503
POST	`/sign`	Ingesta de la secuencia cinemática de 16 fotogramas (matriz de 16×225 flotantes normalizados) y retorno del estado de validación con la glosa detectada.	200, 400, 503
POST	`/activity`	Notificación de movimiento gestual activo. El aviso restablece el temporizador de inactividad (del orden de 4 s) y aplaza el cierre del enunciado.	200, 503
POST	`/utterance/end`	Cierre forzado de la frase en curso, envío de las glosas acumuladas al servicio semántico y limpieza del búfer.	200, 503

`GET /health` no altera estado. El cuerpo de respuesta indica, al menos, si el clasificador local está listo y si el traductor remoto responde. `GET /state` describe el búfer de glosas pendientes y el español (o ausencia de español) del último cierre. `POST /activity` no transporta cuerpo útil: el efecto es únicamente temporal sobre el reloj de pausa. `POST /utterance/end` culmina el enunciado aun cuando el temporizador de inactividad no haya vencido.

Contrato POST `/session`

La apertura de sesión declara la lateralidad del señante. El valor `false` corresponde a ejecución diestra; `true` invierte el eje horizontal de los puntos de pose y manos antes de la clasificación. Una petición válida recibe `200 OK` y un cuerpo de confirmación con la lateralidad efectiva.

Petición:

```
POST /session HTTP/1.1
Host: 127.0.0.1:8765
Content-Type: application/json

{
  "left_handed": false
}
```

Respuesta `200 OK`:

```
{
  "ok": true,
  "left_handed": false
}
```

Contrato POST `/sign`

La ruta recibe una seña ya recortada. El campo `frames` es la secuencia cinemática: dieciséis vectores de 225 flotantes normalizados (33×3 de pose, 21×3 de mano izquierda y 21×3 de mano derecha). El campo `source_fps` declara la cadencia de muestreo de origen, de modo que el motor interprete la duración temporal de la seña con independencia del reloj del cliente.

Petición (secuencia abreviada):

```
POST /sign HTTP/1.1
Host: 127.0.0.1:8765
Content-Type: application/json

{
  "source_fps": 24,
  "frames": [
    [0.12, 0.41, -0.03, 0.18, 0.40, 0.00],
    [0.12, 0.42, -0.03, 0.19, 0.41, 0.00]
  ]
}
```

En el contrato completo, cada uno de los 16 elementos de `frames` posee longitud 225. Un arreglo vacío o una dimensionalidad distinta de 16×225 constituye error de validación (`400 Bad Request`).

Respuesta `200 OK` con seña aceptada:

```
{
  "status": "accepted",
  "gloss": "HOLA",
  "confidence": 0.91,
```

```
  "top3": [
    {"gloss": "HOLA", "confidence": 0.91},
    {"gloss": "BIEN", "confidence": 0.04},
    {"gloss": "GRACIAS", "confidence": 0.02}
  ],
  "utterance_closed": false
}
```

El campo `status` admite `accepted` o `rejected`. El valor `rejected` corresponde a una secuencia que no supera el umbral de confianza o las reglas de validez de la seña; en ese caso `gloss` queda vacío o nulo y `utterance_closed` permanece en `false`. El arreglo `top3` ordena las tres clases de mayor probabilidad. El booleano `utterance_closed` vale `true` solo cuando el motor, tras esta ingesta, considera cerrado el enunciado y dispara el mismo efecto de `POST /utterance/end` (despacho semántico y búfer vacío). En el flujo habitual de una seña intermedia el valor es `false`: la glosa ingresa al búfer y el enunciado sigue abierto.

5.2.2. API del servicio semántico remoto

La arquitectura local se comunica con el microservicio semántico en la nube mediante una API REST sobre HTTPS. Cada petición es independiente: el contrato no incluye memoria de turnos previos, es *serverless*. El canal directo corresponde a la traducción de glosas LSA a una oración en español rioplatense. El canal inverso corresponde a la traducción de una oración en español a una secuencia de glosas con orden *Topic-Comment*, apta para la animación del avatar.

El Cuadro 5.2 resume los endpoints del diseño.

Cuadro 5.2: Endpoints del servicio semántico remoto.

Método	Ruta	Función
GET	`/health`	Consulta de disponibilidad: indica si los modelos de lenguaje (SLM) residen en memoria RAM y si el servicio admite inferencia.
POST	`/translate`	Canal directo (LSA → español rioplatense). Recibe una secuencia de glosas y devuelve una oración cohesiva.
POST	`/hearing`	Canal inverso (español → LSA Topic-Comment). Recibe el texto del interlocutor oyente y devuelve la secuencia de glosas. Esas glosas alimentan después el servicio de archivos del avatar.

Contrato de `POST /translate` (canal directo)

La petición transporta las glosas reconocidas en el edge como una única cadena, con tokens en mayúsculas separados por espacios. La respuesta, en caso de éxito (código 200), incluye la oración en español, el identificador del SLM y el tiempo de inferencia en milisegundos (`ms`).

Petición:

```
{
  "glosses": "APELLIDO MIO PIÑON"
}
```

Respuesta (200 OK):

```
{
  "spanish": "Mi apellido es Piñon.",
  "glosses": "APELLIDO MIO PIÑON",
  "model_used": "qwen2.5-1.5b.gguf",
  "ms": 412.3,
  "queued_ms": 0.0
}
```

Contrato de `POST /hearing` (canal inverso)

La petición transporta la oración en español rioplatense, ya transcripta en el edge. La respuesta, en caso de éxito (código 200), incluye la secuencia de glosas en una cadena de tokens en mayúsculas, el SLM empleado y el tiempo de inferencia.

Petición:

```
{
  "spanish": "Hola, ¿cómo estás?"
}
```

Respuesta (200 OK):

```
{
  "glosses": "HOLA VOS COMO",
  "spanish": "Hola, ¿cómo estás?",
  "model_used": "qwen2.5-1.5b.gguf",
  "ms": 380.1,
  "queued_ms": 0.0
}
```

El campo `queued_ms` registra la espera en cola si el SLM atiende otra inferencia; no forma parte del texto traducido. Un código 503 indica que el modelo no está listo en RAM. El endpoint `GET /health` permite al *edge* diferir el encendido del clasificador local hasta que el atributo `ready` sea verdadero.

5.2.3. API del servicio de archivos del avatar

El canal inverso no anima por sí solo. Un segundo servicio remoto entrega el archivo de movimiento de cada glosa. No traduce y no carga un modelo de lenguaje. El nombre del archivo, sin la extensión, es la glosa ya normalizada. El puerto, la imagen y la plataforma de ese servicio quedan en la Sección 6.4.1.

El Cuadro 5.3 resume las rutas. La de salud responde JSON. La del archivo responde el binario, no un documento JSON.

ILSA arma el nombre antes de la solicitud. El resultado es una sola cadena de letras mayúsculas y números, sin tildes, sin espacios y sin otros signos. Las reglas se aplican en este orden:

Cuadro 5.3: Endpoints del servicio de archivos del avatar.

Método	Ruta	Función
GET	`/health`	Disponibilidad del proceso. Responde 200 con `{"status": "ok"}`. No comprueba si una glosa existe.
GET	`/tposes/{glosa}.pose`	Archivo de esa glosa. El nombre debe coincidir de forma exacta, extensión incluida. 200 y cuerpo binario si existe. 404 si no existe.

1. Toda la glosa pasa a mayúsculas.
2. Queda la forma masculina. El sufijo `_A` o `_RA` se quita. Por ejemplo: `ABOGADO_A` queda `ABOGADO`. `PROFESOR_RA` queda `PROFESOR`.
3. Las vocales con tilde pasan a la vocal simple. `CAFÉ` queda `CAFE`.
4. Se borran espacios, guiones bajos, guiones y cualquier carácter que no sea letra o número. Si una seña tiene más de una glosa, los nombres se concatenan sin separador. Hoy y ahora se piden como `HOYAHORA.pose`, no como dos archivos.
5. La letra `Ñ`, cuando esa es toda la glosa, se pide como `ENIE`. Una `Ñ` dentro de la palabra pasa a `N`. `ESPAÑA` queda `ESPANA`.
6. La glosa de parentesco `PAPA` se pide como `PADRE`. El archivo `PAPA.pose` corresponde al alimento. Para el parentesco, ILSA usa `PADRE`.

En el motor local, el modo oyente agrega dos operaciones sobre el mismo contrato. `POST /hearing` recibe la oración y un indicador de cierre. ILSA consulta el canal inverso y, con las glosas ya obtenidas, pide los archivos. La respuesta trae la oración, la cadena de glosas y el clip. `POST /avatar` recibe solo la cadena de glosas y repite la descarga, sin una traducción nueva. En modo señante ambas rutas responden 400. La dirección del servicio, la caché local y el tratamiento de un nombre sin archivo se describen en la Sección 6.4.1. La búsqueda de una glosa compuesta y el deletreo quedan en la Sección 5.3.

5.3. Diseño de la representación gestual inversa

El canal oyente recibe del traductor inverso una lista de glosas en mayúsculas. Esa lista, por sí sola, es texto: la persona sorda todavía no ve la seña. El diseño convierte cada glosa en el movimiento correspondiente, tomado de un archivo que ya contiene la seña medida sobre un video de referencia. El archivo no guarda una malla ni las rotaciones de un muñeco. Guarda, fotograma a fotograma, la posición de puntos articulares (*landmarks*): hombros, codos, muñecas, las dos manos y un contorno de cara. El servicio de la Sección 5.2.3 entrega ese archivo. Lo que sigue fija cómo se prepara antes de mostrarlo, cómo se une una seña con la siguiente y qué ocurre cuando la glosa no tiene archivo. La tasa común, el peso del suavizado, el tamaño del lienzo, el umbral de confianza, la longitud del puente y la duración de la entrada y de la salida quedan en la Sección 6.4.4.

5.3.1. Clip de referencia

El archivo de cada glosa funciona como el negativo de la seña. Para cada fotograma del video de origen conserva tres cosas: la posición de los puntos en píxeles de esa imagen, la cantidad de fotogramas por segundo y una confianza entre 0 y 1. La confianza dice cuán seguro está el estimador de haber visto ese punto. Esos datos quedan a la resolución del video original, sin achicarlos de antemano.

La escala al tamaño del lienzo se aplica en el momento de dibujar, y no dentro del archivo. El clasificador, en cambio, guarda las coordenadas como fracción del ancho y del alto, porque a la red le alcanza la forma del gesto. Si ese mismo número fraccionario se archivara para el avatar, el visor lo leería como una cantidad de píxeles: un valor 0,5 pasaría a significar medio píxel, y el cuerpo quedaría comprimido contra la esquina del cuadro. El diccionario animable archiva por eso píxeles de imagen. La reducción a un tamaño común ocurre después, cuando ya se conoce el recuadro en el que la seña se va a mostrar.

El archivo crudo tampoco se recorta. Un video de diccionario muestra a la persona desde antes de la seña: levanta la mano, realiza el gesto y vuelve a una postura de reposo. El significado está en el tramo del medio, que aquí se llama núcleo. El levantamiento inicial es la preparación y el regreso al reposo es la retracción. El instante en que empieza el núcleo y el instante en que termina se anotan en un índice aparte, con un tiempo de entrada y un tiempo de salida por glosa. La reproducción lee ese índice. El negativo queda entero, de modo que un cambio del recorte no obliga a estimar el video otra vez.

Al dibujar no entran todos los puntos que el estimador produjo. MediaPipe Holistic marca 33 puntos de cuerpo, 468 de cara y 21 en cada mano. El diseño conserva el torso, de la cintura hacia arriba, y las dos manos completas. Las piernas se apagan, porque en este recuadro la seña se lee de medio cuerpo. Dentro de los 33 puntos de cuerpo también hay una cara gruesa y unas puntas de dedos muy simples. Esos puntos se apagan en el dibujo. Si se trazan junto con los 21 puntos de cada mano y con el contorno fino de la cara, el espectador ve una mano encima de la otra y una cara encima de la otra.

La posición del cuerpo expresada en metros, que el estimador entrega además de la posición en la imagen, se deja de lado por una razón distinta. Esa posición en metros no incluye las 21 articulaciones de la mano. Usarla como si fuera el esqueleto completo deja las manos pegadas al pecho aunque en el video estén junto a la cara. La lectura del avatar es frontal, y lo que importa es el lugar de cada punto en el plano de la imagen. La profundidad de cada mano, además, está medida respecto de su propia muñeca y en una escala mucho menor que el desplazamiento horizontal y vertical. Es un dato interno del estimador, y no una distancia en metros hasta la cámara.

5.3.2. Preparación y empalme

Dos archivos recién descargados todavía no pueden reproducirse uno detrás del otro. Cada video trae su propia velocidad, su propio encuadre y sus propios huecos allí donde el estimador perdió una mano. El diseño obliga a que todo clip atraviese la misma preparación, siempre en el mismo orden. El orden importa porque cada paso deja el material en condiciones para el siguiente: un hueco que no se rellena antes del empalme se convierte, en el empalme, en un salto.

Cadencia común. Los videos de origen no comparten necesariamente la misma cantidad de fotogramas por segundo. Si un clip trae 24 y el siguiente trae 30, una espera de seis

fotogramas dura más en el primero que en el segundo, y el empalme pierde la noción de tiempo. Antes de cualquier otro retoque, todos los clips se llevan a una misma tasa. A partir de ahí, un fotograma significa la misma duración en todas las señas.

Recorte del núcleo. La reproducción usa el índice de entrada y de salida, y deja fuera la preparación y la retracción. Si se mostrara el video completo, quien mira vería a la persona llegar a la seña y retirarse de ella, y el final de una glosa se confundiría con el comienzo de la otra. El núcleo, que es el tramo donde la configuración y el movimiento distinguen esa seña de las demás, es lo único que pasa a la secuencia visible.

Relleno de huecos breves. Cuando el estimador deja de ver una mano durante unos pocos fotogramas, suele escribir el punto en el origen, es decir en la coordenada $(0, 0)$, o dejar un vacío entre dos fotogramas válidos. Si ese vacío es corto y está rodeado de fotogramas en los que la mano sí se vio, el diseño completa la posición con los vecinos válidos. El hueco largo, en el que la mano realmente no participa de la seña, no se inventa: lo resuelve el paso siguiente.

Mano en espera. Hay señas de una sola mano. En esos clips, el estimador puede copiar la mano visible sobre el lugar de la mano ausente, de modo que las dos manos quedan casi en el mismo sitio, o puede dejar la mano ausente en el origen. El diseño distingue esos dos casos de una mano que sí participa. Si las dos manos coinciden en una seña de una sola mano, la copia se retira. La mano que no participa se estaciona al costado del torso, y el antebrazo se dibuja como una línea continua desde el codo hasta la muñeca, para que la muñeca no aparezca quebrada en un ángulo que la seña no tiene. Esa postura de costado es la espera de la mano, y es distinta de la espera de todo el cuerpo, que se describe más abajo.

Suavizado. Aunque la mano esté bien seguida, la posición estimada tiembla de un fotograma al siguiente. El diseño mezcla cada medición nueva con la pose ya aceptada en el fotograma anterior, por un promedio exponencial. Un peso alto sigue al estimador con mayor fidelidad y deja pasar el temblor. Un peso bajo alisa de más y retrasa el gesto respecto del núcleo. El peso es el mismo para todo el diccionario. Su valor, y la ecuación, están en la Sección 6.4.4.

Lienzo común. Los videos no comparten resolución ni tamaño de cuerpo. Si cada clip se dibujara a su escala original, al pasar de una glosa a la otra el esqueleto cambiaría de tamaño en medio de la frase. Después del suavizado, el cuerpo se lleva a un recuadro común y se escala hasta un mismo ancho de hombros. Dos señantes de distinta talla, y dos tomas de distinto encuadre, ocupan entonces el mismo espacio, y el empalme no produce un salto de escala.

Recién después de esos seis pasos el clip está en condiciones de unirse a otro. El dato que más daño hace, si se le deja llegar, es la coordenada nula. Supóngase que el último fotograma útil de `HOLA` tiene la muñeca en un punto interior del cuadro, y que el primero de `MAMÁ` trae esa misma muñeca en $(0, 0)$ porque el estimador no la vio. Una interpolación de unos pocos fotogramas entre esos dos puntos dibuja la mano en viaje hacia la esquina, como si la seña la llevara hasta allí. El relleno y la mano en espera existen para que el empalme reciba solo coordenadas válidas. Entre el último fotograma del núcleo de una

glosa y el primero del núcleo de la siguiente, el diseño inserta un puente: una serie breve de fotogramas nuevos, que no estaban en ninguno de los dos archivos. La serie es más larga cuando las muñecas de un extremo y del otro quedan más lejos, para que el recorrido no sea un salto. La curva con la que se reparte ese recorrido se llama *smoothstep*: el punto sale despacio, acelera en la mitad y llega despacio, de modo que la unión no arranca con un tirón. Cada punto viaja en línea recta, en el plano de la imagen, desde donde está hasta donde tiene que estar. El archivo no contiene ángulos de huesos, y por eso el puente no gira una articulación: traslada los puntos.

Ese puente es un recurso gráfico. En la lengua, la coarticulación es otra cosa: el final de una seña ya anticipa la configuración de la siguiente. El puente no modela ese anticipo. Toma la pose final de un núcleo ya recortado y la pose inicial del núcleo siguiente, y cubre el espacio que hay entre ambas para que el ojo no vea un corte. El interior de cada seña llega al empalme tal como quedó después de la preparación. La preparación de un clip tiene un costo de cálculo, y la descarga del archivo tiene un costo de red. Una vez que un clip quedó preparado, el diseño lo conserva en una caché en memoria. Si la misma glosa vuelve a aparecer en el enunciado, o en un enunciado posterior de la misma sesión, se reutiliza esa copia: no se vuelve a preparar y no se vuelve a pedir por la red. Además, la secuencia visible tiene un techo de 24 glosas y de 1200 fotogramas. Por encima de ese techo, un texto largo ocuparía el turno de la llamada con una animación que no termina. El diseño corta la secuencia en el techo. El Cuadro 5.4 resume estas previsiones. El detalle de cada una está en los párrafos anteriores.

Cuadro 5.4: Previsiones del diseño frente a fallos del estimador y del léxico.

Situación	**Previsión**
La mano desaparece durante unos pocos fotogramas y el estimador escribe $(0, 0)$.	Completa el hueco con los fotogramas vecinos en los que la mano sí se vio.
En una seña de una sola mano, las dos manos quedan en el mismo lugar.	Retira la copia y estaciona ese brazo al costado del torso.
La confianza de un punto es demasiado baja.	Omite ese punto en el trazo.
La posición estimada tiembla de un fotograma al otro.	Mezcla la medición nueva con la pose anterior, con un peso fijo.
La muñeca final de una glosa queda lejos de la muñeca inicial de la siguiente.	Inserta un puente breve de fotogramas nuevos entre los dos núcleos, sin alterar el interior de cada seña.
La glosa no tiene archivo.	La deletrea con los clips del alfabeto manual.
Tampoco están los archivos de las letras.	Deja el esqueleto en la postura de espera y avisa en la interfaz.
El enunciado supera 24 glosas o 1200 fotogramas.	Corta la secuencia en ese techo.

5.3.3. Presentación en la videollamada

Quien está del otro lado de la llamada ve la imagen que el navegador entrega como cámara. El diseño sustituye esa imagen por un lienzo partido en dos mitades de la misma altura. La mitad izquierda muestra la cámara de la persona, recortada para llenar su mitad sin estirar la figura. La mitad derecha muestra el esqueleto. Las dos mitades comparten la altura para que el intérprete y la persona queden a la misma escala.

Mientras no llega una lista de glosas, el esqueleto permanece en espera y no adelanta ninguna seña. La postura de espera se toma de un tramo en el que la persona todavía no empezó el gesto, y se escala al mismo ancho de hombros del lienzo. El antebrazo queda casi horizontal, levemente hacia abajo, y la mano sigue esa línea. La cara de la espera es un óvalo con cejas, ojos, nariz y boca, suficiente para situar la cabeza respecto de las manos.

Cuando la lista de glosas ya está resuelta y los fotogramas están empalmados, la reproducción no avanza un fotograma del clip por cada imagen que publica la llamada. La llamada puede publicar 24 imágenes por segundo mientras el clip quedó preparado a otra tasa. Si se avanzara al ritmo de la llamada, la seña saldría más lenta o más rápida que el video de origen. El diseño mide el tiempo transcurrido y elige el fotograma del clip que corresponde a ese tiempo.

Esa reproducción tiene tres tramos. En la entrada, un tramo breve a la tasa del clip, la postura de espera se transforma en el primer fotograma de la seña, y el contorno de cara de la seña aparece de a poco. En el tramo central corre la secuencia ya preparada, con los puentes entre glosas ya incluidos en la lista de fotogramas. En la salida, de la misma duración que la entrada, el último fotograma vuelve a la espera, el contorno de la seña se apaga y queda la cara simple de la espera. La curva de la entrada y de la salida es la misma *smoothstep* del puente, de modo que el esqueleto no aparece ni desaparece de golpe. Al terminar, el clip se suelta y el esqueleto queda otra vez en espera, listo para la frase siguiente.

Hacia el lienzo no viajan los cientos de puntos del archivo. Viaja solo lo que hace falta para leer la seña en una mitad de cuadro: ocho puntos de pose, que son los dos hombros, los dos codos, las dos muñecas y las dos caderas; los 21 puntos de cada mano; y un contorno de cara formado por el óvalo, las cejas, los ojos, la nariz y los labios. En el recuadro pequeño de la llamada, esa malla se ve como una mancha densa y aporta poco a la configuración de los dedos, que es lo que la persona sorda necesita distinguir. Un punto de confianza baja se envía como ausente. El trazo lo saltea, en lugar de unir el punto anterior con una posición en la que el estimador casi no cree. Cada dedo lleva un color fijo, y ese color es el mismo en las dos manos: pulgar rojo, índice naranja, medio amarillo, anular verde y meñique azul. La palma es blanca. El color permite seguir un dedo cuando las manos se cruzan delante del cuerpo y, sin él, los trazos de una y otra mano se confundirían. La orientación también está fijada. En una toma de frente, la mano derecha de la persona aparece del lado izquierdo de la imagen, porque la cámara la ve de frente. El lienzo respeta esa orientación y no da vuelta el esqueleto para que la mano hábil quede del lado que el espectador llamaría derecho. La miniatura propia de la videollamada, en cambio, suele mostrarse espejada, como un reflejo. El diseño exige que esa persona y quien está del otro lado vean lo mismo: la cámara en la mitad izquierda y el esqueleto, sin invertir, en la mitad derecha. El mecanismo que deshace el espejo solo en la vista local queda en la Sección 6.4.4.

5.3.4. Contingencia léxica

El nombre con el que se pide cada archivo sigue las reglas de la Sección 5.2.3: mayúsculas, sin tildes y sin espacios. Aquí el problema es otro. Una lista de glosas puede nombrar una seña con una palabra o con varias, y también puede nombrar algo que el diccionario no tiene.

La búsqueda prueba primero el grupo más largo. Toma tres glosas seguidas y pregunta si existe un archivo para ese grupo. Si no existe, prueba las dos primeras. Si tampoco

existe, prueba una sola. El motivo es que una expresión como `BUENAS TARDES` puede ser una única seña, con un único archivo, y partirla de entrada en `BUENAS` y `TARDES` mostraría dos gestos distintos del que corresponde. Un dígito aislado, como el carácter `1`, se busca por el nombre de ese número, `UNO`, porque el diccionario nombra el número y no el signo con el que se escribe.

Cuando ni el grupo ni la palabra tienen archivo, el diseño deletrea la palabra con los clips del alfabeto manual. Así, un nombre propio como `FRANCISCO`, si no tiene seña propia, se muestra como la secuencia `F`, `R`, `A`, `N`, `C`, `I`, `S`, `C`, `O`. En ese alfabeto, `CH` es una sola letra, y no la letra `C` seguida de la letra `H`. `Ñ` también es una sola letra. Esta regla pertenece al deletreo. Es independiente de la regla del nombre de archivo, que solo decide cómo se escribe la cadena en el pedido.

Si falta el diccionario entero, faltan también las letras. En ese caso no hay con qué deletrear. El esqueleto permanece en la postura de espera, y la interfaz informa que el diccionario no está disponible. La palabra no se omite sin aviso, y tampoco se reemplaza por un movimiento aproximado: lo único que el lienzo puede mostrar es un clip que existe.

5.4. Patrones de diseño de software aplicados

El diseño separa las responsabilidades del cliente local, de la extensión y del servicio remoto. Cada patrón acota un problema distinto: la política de admisión de glosas, la interfaz única hacia el motor, la conversión entre contratos y el modo de la sesión. El efecto de cada decisión queda en el componente dueño. La red no filtra rebotes, la extensión no llama a la nube y el servicio remoto no altera el estado de la sesión local. La referencia de estos patrones es (Gamma et al., 1994).

- **Estrategia (*Strategy*), en el filtro de repetición.** El clasificador propone una glosa y una confianza. No decide si esa glosa entra al enunciado. Esa decisión queda en un objeto de política, independiente de la red. La política distingue tres categorías del léxico: los dígitos admiten repetición, las letras admiten hasta dos apariciones consecutivas y el resto del vocabulario rechaza la repetición inmediata. El acumulador de enunciado consulta la política y conserva el reloj de pausa Un cambio de la regla de repetición no altera el modelo ni el contrato remoto.

- **Fachada (*Facade*), en la API local de ILSA.** La extensión no invoca por separado al clasificador, a la compuerta, al acumulador ni al cliente de la nube. El motor expone un único contrato HTTP en el bucle local: sesión, ingesta de la seña, aviso de actividad, cierre de enunciado, estado y diagnóstico. La fachada oculta el orden interno del pipeline y concentra el acceso del navegador.

- **Adaptador (*Adapter*), en dos fronteras.** La extensión adapta los medios de Google Meet al contrato local. La cámara y el lienzo de la llamada se presentan como fotogramas y subtítulos, y el audio o el texto del oyente se presentan como una oración. En la otra frontera, ILSA adapta el enunciado cerrado al contrato del servicio remoto: una cadena de glosas en `/translate`, o una oración en `/hearing`. Con las glosas del canal inverso, ILSA arma el nombre de archivo y consulta `/tposes`. La respuesta remota vuelve a entrar por ILSA. Ni el traductor ni el servicio de archivos conocen Meet ni la extensión.

- **Estado (*State*), en el modo de sesión.** La sesión admite dos modos excluyentes: señante y oyente. En modo señante el motor acepta señas y cierra enunciados hacia el canal directo. En modo oyente acepta el texto del interlocutor, consulta el canal inverso y pide los archivos de seña. La misma fachada cambia de comportamiento según el modo, sin una segunda extensión. El temporizador de pausa no es un estado del clasificador: pertenece al acumulador y solo declara el cierre del enunciado.

5.5. Diseño conceptual de la arquitectura escalable en la nube

El módulo semántico queda fuera de la computadora del usuario. El cliente procesa la cámara, la extracción de puntos y la clasificación, y envía el texto del turno en curso al servicio remoto.

Esa partición admite, ante varias videollamadas simultáneas, una infraestructura elástica. Este trabajo la fija como proyección de producción. Las pruebas del prototipo no la despliegan: corren sobre una instancia única de Cloud Run, de Google Cloud. La subsección siguiente describe la proyección.

5.5.1. Especificación para entornos de producción concurrente

Para garantizar alta disponibilidad y bajos tiempos de respuesta por turno frente a múltiples usuarios concurrentes, se proyectó una arquitectura modular basada en microservicios contenerizados y auto-escalado elástico. La topología de recursos y el flujo de balanceo se describen en la Figura 5.5.

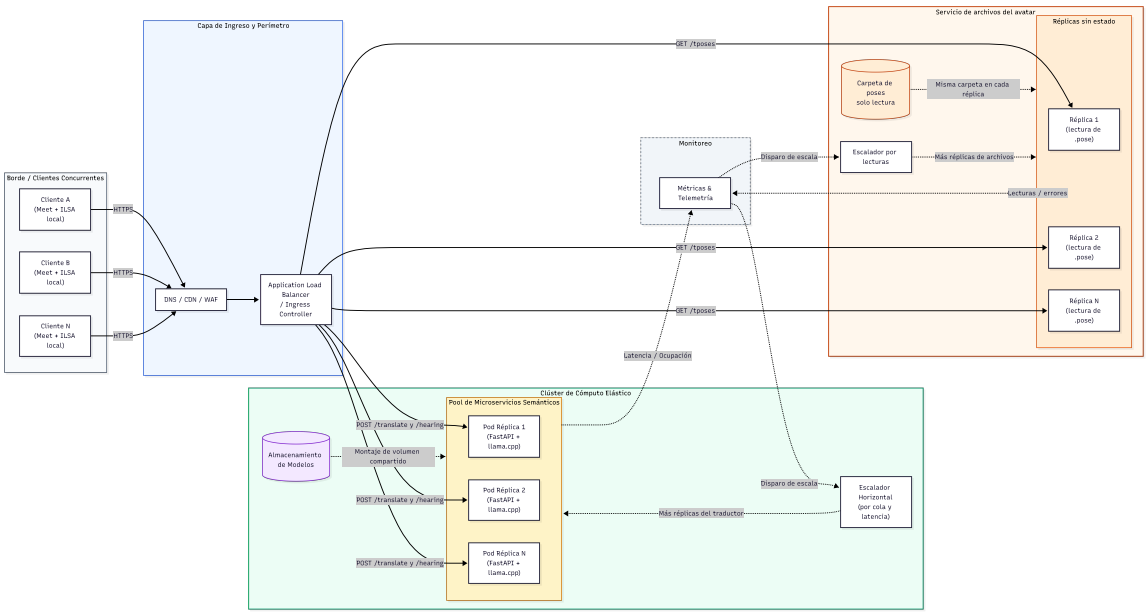


Figura 5.5: Diseño conceptual de la infraestructura en la nube: balanceo de carga y clúster elástico de inferencia semántica con escalado horizontal.

El diseño contempla tres capas principales de infraestructura:

Capa de acceso perimetral y balanceo: Las solicitudes HTTPS enviadas por las computadoras clientes son recibidas por una capa perimetral que incluye resolución DNS, certificados de seguridad TLS y filtrado de tráfico básico para prevenir saturaciones maliciosas. Detrás de esta capa opera un balanceador de carga (*API Gateway* o *Application Load Balancer*):

- Recibe y valida los esquemas JSON de las rutas `/translate` y `/hearing`.
- Distribuye las peticiones hacia las réplicas del microservicio semántico mediante un criterio de menor número de conexiones activas (*Least Connections*) para evitar sobrecargar instancias que se encuentren resolviendo enunciados largos.

Clúster de cómputo y escalado elástico: El microservicio semántico se empaqueta en imágenes de contenedor Docker. Estos contenedores se despliegan sobre un clúster gestionado:

- **Almacenamiento compartido de modelos:** Los archivos binarios cuantizados en formato GGUF (Q4_K_M) se almacenan en un volumen persistente de solo lectura montado en cada instancia para evitar transferencias repetitivas de red al iniciar nuevos contenedores.
- **Auto-escalado por demanda:** Mediante un controlador de escalado horizontal, el clúster monitorea la longitud de la cola de peticiones y los tiempos de generación de respuesta. Cuando la cantidad de turnos simultáneos supera el umbral operativo de las réplicas activas, el orquestador provisiona automáticamente nuevos contenedores para absorber el pico de uso.
- **Eficiencia de hardware y costos:** Gracias a la cuantización a 4 bits (GGUF Q4_K_M), cada réplica requiere menos de 2 GB de memoria RAM y puede ejecutarse sobre procesadores estándar en la nube, lo que quita la necesidad de servidores dedicados con placa de video.

Monitoreo y telemetría: Se prevé un sistema de recopilación de métricas (como Prometheus y Grafana o AWS CloudWatch) para supervisar la latencia por oración, la memoria ocupada por contenedor y la tasa de errores de red lo que permite calibrar con precisión las reglas de escalado.

5.5.2. Instancia de prueba en Cloud Run

La arquitectura de la figura anterior exige un clúster con balanceo y auto-escalado. Ese esquema queda fuera de las pruebas de este Trabajo Final, en coincidencia con la restricción RC-02. En su lugar, el motor local ILSA consulta una instancia del servicio semántico publicada en Cloud Run. La instancia expone las rutas `/translate` y `/hearing` y concentra los dos canales en un único proceso. El comportamiento bajo carga concurrente y el auto-escalado no entran en la validación del prototipo.

5.5.3. Servicio de archivos del avatar

Los archivos de seña no siguen el clúster del modelo de lenguaje. Cada petición es una lectura, no una generación. No hay cola de inferencia ni placa de video. El contrato de esa lectura es el de la Sección 5.2.3.

Para un despliegue de producción, la proyección es un conjunto de réplicas sin estado detrás del balanceador de la plataforma. Todas leen la misma carpeta, de solo lectura. Un paso posterior puede mover los archivos a un almacén de objetos y dejar el proceso como un lector delgado, con una red de contenido delante para las glosas repetidas. La ruta de salud sigue aparte de la ruta del archivo. La caché del cliente permanece como primer recurso. Esta proyección no forma parte de la instancia única con la que se prueba el prototipo. El despliegue de esa instancia de prueba se describe en la Sección 6.4.1.

Capítulo 6

Implementación y Desarrollo

6.1. Implementación del subsistema de reconocimiento gestual

El subsistema de clasificación convierte secuencias cinemáticas en glosas de un vocabulario de 100 clases.

6.1.1. Preprocesamiento de datos

Pipeline de captura e ingesta con MediaPipe Holistic. La extracción de datos utiliza MediaPipe Holistic para obtener 33 puntos de pose y 21 puntos por cada mano, conformando un descriptor instantáneo de 225 coordenadas ($33 \times 3 + 21 \times 3 + 21 \times 3$).

Normalización geométrica y corrección de aspecto. Cada fotograma se centra en el punto medio de los hombros y se escala por la distancia inter-hombros. Para corregir la distorsión producida por la relación de aspecto entre cámaras ($4:3$ vs. $16:9$), se implementó un algoritmo de relleno con bandas a 853×480 previa extracción, evitando deformaciones en el espacio cartesiano.

6.1.2. Preprocesamiento y arquitectura neuronal TinySkeletonClassifier

El flujo de entrada se procesa mediante tres etapas: interpolación lineal de fotogramas nulos, recortes de silencio inicial/final y submuestreo uniforme a una longitud fija de $T = 16$ fotogramas.

La red neuronal *TinySkeletonClassifier* está compuesta por:

1. Dos capas de convolución 1D temporal (núcleo 3, 128 canales, BatchNorm, ReLU y Dropout 0,2).
2. Codificación posicional sinusoidal y un Transformer Encoder de 2 capas con 4 cabezas de atención.
3. Capa de agregación por atención (*Attention Pooling*) que asigna un peso ponderado a cada instante según su relevancia informativa.
4. Capa lineal final hacia los logits de clasificación.

6.2. Implementación del subsistema de traducción semántica bidireccional

6.2.1. Selección y especificación de la suite de modelos de lenguaje

Los canales semántico directo (LSA → Español) e inverso (Español → LSA) entrenan la misma suite de modelos de lenguaje pequeños, entre 0,5B y 3B parámetros:

- **Familia Qwen2.5:** variantes de instrucciones de 0,5B, 1,5B y 3B.
- **Familia Llama 3.2 (Dubey et al., 2024):** variante de instrucciones de 1B.
- **Familia SmolLM2 (Allal et al., 2024):** variante de instrucciones de 1,7B.

El entrenamiento puede recorrer la lista completa o limitarse a un subconjunto. El servicio en la nube no carga los cinco modelos. Cada canal arranca con Qwen2.5 de 1,5B y, si ese archivo no está disponible, con Llama 3.2 de 1B. SmolLM2 y las variantes Qwen de 0,5B y 3B permanecen en la suite de entrenamiento y de exportación. No tienen un lugar reservado en el arranque. Otro archivo cuantizado en la carpeta del canal entra solo como candidato posterior.

6.2.2. Canal directo (LSA → Español)

El canal directo recibe las glosas de un turno cerrado y devuelve una sola oración en español rioplatense. El procedimiento tiene cuatro tramos: preparación del par glosas–oración, ajuste de bajo rango, consigna de traducción y generación sobre el modelo cuantizado.

Preparación del corpus. Cada ejemplo asocia una secuencia de glosas con una oración. Antes del ajuste, las letras sueltas y consecutivas se funden en un token con mayúscula inicial, y los dígitos sueltos y consecutivos se funden en un número. Tres o más letras iguales seguidas se reducen a dos. Una letra aislada X o Ñ permanece en mayúscula. Así, el modelo no aprende a pegar el deletreo: recibe el nombre o la cifra ya compactados. Una ráfaga de letras sin seña de separación, como el deletreo continuo de un topónimo de varias palabras, queda en un solo token.

El conjunto se divide en un 85 % de entrenamiento y un 15 % de evaluación, con semilla fija. Cada ejemplo de entrenamiento forma un diálogo de tres turnos. El primero fija la consigna de sistema. El segundo, el del usuario, antepone la etiqueta «Glosas:» a la secuencia ya unida. El tercero, el del asistente, contiene la oración esperada. Cada familia conserva su propia plantilla de diálogo. La longitud máxima de cada ejemplo durante el ajuste es de 512 tokens y los ejemplos no se concatenan dentro de la misma secuencia. La carga del modelo admite un contexto de hasta 2048 tokens y, solo en esta etapa, guarda los pesos en 4 bits.

Ajuste de bajo rango. La adaptación usa LoRA (Hu et al., 2022), con rango $r = 16$ y factor de escala $\alpha = 32$. Las matrices cubren las cuatro proyecciones de la atención (consulta, clave, valor y salida) y las tres proyecciones de la red de avance (compuerta, expansión y contracción). La tabla de incrustaciones y la proyección final del vocabulario

permanecen congeladas. No hay abandono de unidades ni sesgo adicional en los adaptadores. Para contener la memoria, el ajuste conserva solo algunos estados intermedios y recalcula el resto.

El régimen por defecto recorre cinco épocas. El modelo de 3B usa lotes de 2 ejemplos y acumula el gradiente durante 8 pasos; los demás usan lotes de 4 y acumulan durante 4 pasos. La tasa de aprendizaje es 3×10^{-4} para el modelo de 3B y 5×10^{-4} para el resto. El optimizador es Adam de 8 bits, con decaimiento de pesos 0,01, diez pasos de calentamiento y un planificador coseno. Si el equipo dispone de *bfloat16*, el cálculo usa esa precisión; en caso contrario, usa *float16*. Una corrida puede reemplazar las épocas por un tope de pasos.

La pérdida no observa todo el diálogo. Una máscara reconoce, en la plantilla de cada familia, el inicio del turno de usuario y el inicio del turno de asistente. La consigna y las glosas quedan fuera del gradiente. El error se calcula solo sobre los tokens de la oración en español.

Consigna de traducción. La consigna obliga a una única oración, sin notas, comillas ni datos ajenos a la entrada. Las reglas que el ajuste debe fijar son las siguientes (Massone & Martínez, 2012):

- El español es rioplatense y usa voseo. La oración incorpora artículos, preposiciones y concordancia verbal. La traducción no es palabra por palabra.
- Los pronombres de entrada son `YO`, `VOS`, `EL`, `NOSOTROS` y `ELLOS`. Sin una marca de género al lado, el sujeto queda tácito.
- La letra `A` inmediata al sustantivo o al pronombre marca femenino, y la letra `O` marca masculino. Un parentesco sin esa letra se vierte en masculino.
- `MIO` y `TUYO` se vierten como posesivos del español.
- `PASADO` o `AYER` exigen pretérito. `FUTURO` exige futuro o una perífrasis. `YA` o `TERMINAR` marcan una acción concluida. Sin marca temporal, el verbo queda en presente.
- `DONDE`, `QUIEN`, `COMO`, `QUE`, `CUANDO` y `CUANTOS` producen una pregunta directa, con signos de apertura y de cierre.
- `NO` junto al concepto o al verbo produce una negación fluida.
- Un token en capitalización de nombre, o una cifra, se copia tal cual. La consigna prohíbe sustituirlo o alterarlo.
- La entrada suele seguir el orden *Topic-Comment* (tiempo, lugar, sujeto, sustantivo, posesivo, numeral, verbo y pregunta). La oración de salida reordena esa secuencia según la sintaxis del español.

Generación. ILSA junta las letras antes de llamar al modelo. Dos o más letras seguidas pasan a un nombre con mayúscula inicial, y dos o más dígitos seguidos pasan a un número. Una letra sola no se compacta, y esta unión no recorta las repeticiones de una misma letra. El modelo recibe esa cadena, con el nombre ya capitalizado, y no el deletreo letra por letra. Si el enunciado completo es solo letras o solo dígitos, ILSA arma el texto en el cliente y no llama al modelo. En los demás casos, el mensaje reúne la consigna de sistema, la ventana de turnos previos de ese cliente y el turno «Glosas:» con la secuencia ya unida. La generación

admite hasta 64 tokens nuevos, con temperatura 0,1 y penalización de repetición 1,0. El canal intenta primero Qwen2.5 de 1,5B. Ante una respuesta vacía o un fallo, pasa a Llama 3.2 de 1B. La misma corrida de entrenamiento, ya con el adaptador fusionado y el binario cuantizado, evalúa el 15 % reservado. Esa evaluación usa temperatura 0, un tope de 24 tokens nuevos y penalización de repetición 1,05, y se detiene ante un salto de línea o la marca de fin de turno. Si varias oraciones del corpus comparten la misma secuencia de glosas, todas cuentan como referencia válida de esa entrada. La corrida registra el acierto exacto, el acierto tras normalizar puntuación y mayúsculas, y las métricas BLEU, ROUGE-L y METEOR, además de la latencia de cada generación.

6.2.3. Canal inverso (Español → LSA)

El canal inverso recibe una oración en español rioplatense y devuelve una sola secuencia de glosas de la LSA. El par de entrenamiento invierte al del canal directo: la entrada es la oración y la respuesta esperada es la secuencia de glosas. El procedimiento recorre la preparación del corpus, el ajuste de bajo rango, la consigna de glosado y la generación sobre el modelo cuantizado.

Preparación del corpus. Cada ejemplo asocia una oración con su secuencia de glosas. Antes del ajuste, las letras sueltas y consecutivas de esa secuencia se funden en un token con mayúscula inicial, y los dígitos sueltos y consecutivos se funden en un número. Tres o más letras iguales seguidas se reducen a dos. Una letra aislada `X` o `Ñ` permanece en mayúscula. El modelo aprende así a emitir el nombre o la cifra ya compactados, y no una lista de caracteres.

El conjunto se divide en un 85 % de entrenamiento y un 15 % de evaluación, con semilla fija. Si varias secuencias de glosas comparten la misma oración, todas cuentan como referencia válida de esa oración. Cada ejemplo de entrenamiento forma un diálogo de tres turnos. El primero fija la consigna de sistema. El segundo, el del usuario, antepone la etiqueta «Español:» a la oración. El tercero, el del asistente, contiene las glosas separadas por espacios. Cada familia conserva su plantilla de diálogo. La longitud máxima de cada ejemplo durante el ajuste es de 512 tokens y los ejemplos no se concatenan dentro de la misma secuencia. La carga del modelo admite un contexto de hasta 2048 tokens y, solo en esta etapa, guarda los pesos en 4 bits.

La consigna de sistema incluye, en la corrida por defecto, ejemplos ya resueltos. Cada uno muestra una oración y la secuencia de glosas en la forma compacta. Una corrida puede omitir esos ejemplos.

Ajuste de bajo rango. El rango, el factor de escala y las capas coinciden con el canal directo: $r = 16$, $\alpha = 32$, las cuatro proyecciones de la atención y las tres de la red de avance. La tabla de incrustaciones y la proyección final del vocabulario permanecen congeladas. No hay abandono de unidades ni sesgo adicional. El ajuste conserva solo algunos estados intermedios y recalcula el resto. Si la familia no expone esas proyecciones con los nombres previstos, la adaptación cubre todas las proyecciones lineales del modelo.

El régimen también coincide. Cinco épocas por defecto. El modelo de 3B usa lotes de 2 y acumula el gradiente durante 8 pasos; los demás usan lotes de 4 y acumulan durante 4 pasos. La tasa de aprendizaje es 3×10^{-4} para el modelo de 3B y 5×10^{-4} para el resto. El optimizador es Adam de 8 bits, con decaimiento de pesos 0,01, diez pasos de calentamiento

y un planificador coseno. Si el equipo dispone de *bfloat16*, el cálculo usa esa precisión; en caso contrario, usa *float16*. Una corrida puede reemplazar las épocas por un tope de pasos.

La máscara de pérdida reconoce el inicio del turno de usuario y el inicio del turno de asistente. La consigna y la oración en español quedan fuera del gradiente. El error se calcula solo sobre los tokens de las glosas.

Consigna de glosado. La consigna exige una única secuencia, separada por espacios, sin notas, comillas ni el prefijo «Glosas:». La traducción no es palabra por palabra: no aparecen artículos, preposiciones ni signos del español. Las reglas que el ajuste debe fijar son las siguientes (Massone & Martínez, 2012):

- Las glosas léxicas van en mayúsculas sostenidas. Un nombre, un apellido o una calle va en capitalización de nombre, como `Maite` o `Moreno`, y nunca letra por letra. Una cifra, como un documento o una cantidad, va en un solo token numérico.
- El orden es tiempo, lugar, sujeto, sustantivo, posesivo, numeral, verbo y, al cierre, la partícula interrogativa o la negación. Tres consignas de referencia son "¿Dónde vas a vivir vos?" → `FUTURO VOS VIVIR DONDE`, "No voy a llevar el cuchillo." → `FUTURO YO CUCHILLO LLEVAR NO` y "Vivo en la calle Moreno." → `CALLE Moreno YO VIVIR_EN`.
- Los pronombres se explicitan aunque el español los omita. "Tengo un cuchillo." → `YO CUCHILLO TENER`. El mapeo es yo → `YO`, vos o tú → `VOS`, él → `EL HOMBRE`, ella → `EL MUJER`, nosotros → `NOSOTROS`, nosotras → `NOSOTROS MUJER`, ellos → `ELLOS HOMBRE` y ellas → `ELLOS MUJER`.
- El género del parentesco usa el sustantivo en masculino más `HOMBRE` o `MUJER`: hermana → `HERMANO MUJER`, hija → `HIJO MUJER`, esposa → `ESPOSO MUJER`. Un sustantivo de persona sin marca de género queda en masculino y suma `HOMBRE` si el español lo explicita.
- `MIO` y `TUYO` cubren los posesivos. "Me llamo X" o "Mi nombre es X" → `YO NOMBRE` más el nombre.
- El pretérito o el copretérito colocan `PASADO` al inicio. El futuro, o la perífrasis "voy a", coloca `FUTURO`. "Ya" coloca `YA`. El presente sin marca no agrega token temporal.
- Las preguntas de dónde, quién, cómo, qué, cuándo y cuántos dejan `DONDE`, `QUIEN`, `COMO`, `QUE`, `CUANDO` o `CUANTOS` al final. La negación deja `NO` al final, junto al verbo.
- "Número de documento" o "DNI" produce `DOCUMENTO NUMERO` más la cifra compacta. "Apellido" produce `APELLIDO` más el nombre. "Calle" produce `CALLE` más el nombre. Esos nombres y esas cifras se copian tal cual.

ILSA no recibe un deletreo. Recibe la secuencia de glosas, y un nombre o un número llega ahí como un solo token.

Generación. El modelo recibe la oración en español. Un nombre propio ya figura en esa oración con mayúscula inicial. La respuesta es una secuencia de glosas. Si el enunciado trae un nombre o un número, el modelo los devuelve compactos: el nombre, en un token con mayúscula inicial, y el número, en un solo token. No los divide en letras. ILSA conserva esa secuencia tal como llega y no la parte en deletreo. Esta llamada no recibe los turnos

previos. La generación admite hasta 64 tokens nuevos, con temperatura 0,1 y penalización de repetición 1,0. El canal intenta primero Qwen2.5 de 1,5B. Ante una respuesta vacía o un fallo, pasa a Llama 3.2 de 1B.

La misma corrida de entrenamiento, ya con el adaptador fusionado y el binario cuantizado, evalúa el 15 % reservado. Esa evaluación usa temperatura 0, un tope de 48 tokens nuevos y penalización de repetición 1,05, y se detiene ante un salto de línea o la marca de fin de turno. Si la salida trae un bloque de razonamiento, ese bloque se descarta y solo queda la primera línea. Antes de puntuar, las letras y los dígitos sueltos de la predicción se compactan con las mismas reglas del corpus. La corrida registra el acierto exacto, el acierto tras pasar a minúsculas y quitar la puntuación, y las métricas BLEU, ROUGE-L y METEOR, además de la latencia de cada generación.

6.2.4. Tratamiento de letras y dígitos sueltos

El léxico clasificado no cubre nombres ni cifras arbitrarias. La unión no queda a cargo de la red.

En el canal directo, ILSA junta las letras antes de enviarlas al modelo. Dos o más letras seguidas forman un nombre con mayúscula inicial, y dos o más dígitos seguidos forman un número. El modelo recibe ese nombre ya capitalizado. Una letra sola no se compacta. Si el enunciado es solo un deletreo o solo dígitos, ILSA resuelve el texto en el cliente y la llamada al modelo no ocurre.

En el canal inverso, el modelo recibe la oración, con el nombre ya capitalizado, y devuelve glosas. Un nombre o un número sale compacto y, en el caso del nombre, con mayúscula inicial. El modelo no lo divide en letras. ILSA recibe esas glosas y no un deletreo.

En el corpus de entrenamiento la misma forma compacta es la que ve el modelo directo y la que debe emitir el modelo inverso. Tres o más letras iguales seguidas se reducen a dos, y una letra aislada X o Ñ permanece en mayúscula. Una ráfaga sin seña de separación, como el deletreo continuo de *Mar del Plata*, queda en un solo token. No hay una regla posterior que parta ese token en las palabras del topónimo.

6.2.5. Cuantización GGUF Q4_K_M y servicio de inferencia

Al terminar cada corrida, el adaptador se fusiona con los pesos base y el resultado se exporta al formato GGUF con cuantización Q4_K_M, por medio de `llama.cpp` (Gerganov, 2023). La evaluación de esa corrida carga el binario cuantizado.

El servicio usa la misma biblioteca de inferencia. Si el enlace desde Python no está disponible, el proceso usa el ejecutable nativo de `llama.cpp`. En ambos casos ninguna capa pasa a una placa de video y la inferencia queda en el procesador. El contexto por defecto es de 2048 tokens cuando el proceso publica un puerto de servicio, y de 4096 tokens en caso contrario. Una configuración explícita puede fijar otro tamaño. Cada canal tiene su propia carpeta. El arranque busca primero Qwen2.5 de 1,5B y después Llama 3.2 de 1B.

Las rutas `/translate` y `/sordo` atienden el canal directo. El cuerpo trae una sola cadena, las glosas del turno, y no una lista. Las rutas `/hearing` y `/oyente` atienden el canal inverso. El cuerpo trae una sola cadena, la oración en español. Ninguna petición incluye el historial. La respuesta repite la cadena recibida, entrega la cadena producida, indica el modelo que respondió, el tiempo de generación, el tiempo de espera en cola, el cliente y la cantidad de turnos retenidos.

Entre la consigna y las glosas actuales, el canal directo puede insertar hasta diez turnos previos del mismo cliente. Un turno del señante se guarda como sus glosas y la oración producida. Un turno del oyente se guarda como la oración oída y una nota fija de contexto, no como glosas. La traducción inversa no recibe esos turnos, aunque el turno oído sí entra en la ventana del canal directo. La petición de traducción no vacía esa ventana.

Formulación analítica para la estimación de memoria RAM en tiempo de inferencia. Esta estimación no forma parte del servicio. Acota la memoria del binario Q4_K_M en el procesador, con la ventana de 2048 tokens del entorno publicado:

$$\text{RAM de inferencia} \approx \mathbf{W} + KV_{\text{fp16}}(ctx = 2048) + \text{Buffer de trabajo} \tag{6.1}$$

donde:

- **W** es el espacio de los tensores cuantizados, del orden de 0,5 bytes por parámetro en `GGUF Q4_K_M`.
- KV_{fp16} es la caché de claves y valores en `fp16` para 2048 tokens (Pope et al., 2023).
- El buffer de trabajo reserva memoria temporal para las activaciones ($2 \times d_{\text{model}} \times ctx \times 2$ bytes).

6.3. Integración en Google Meet y extensión de navegador

La integración con la plataforma Google Meet se implementó como una extensión de navegador desarrollada bajo la especificación Manifest V3 de Chromium (Google Chrome Developers, 2024). La arquitectura física del código distribuye las responsabilidades en un script en segundo plano (*service worker*), un documento oculto (*offscreen*), una página de entorno aislado (*sandbox*) y dos scripts de contenido (*content scripts*) inyectados sobre el dominio de la videollamada (Google Chrome Developers, 2024). La Figura 6.1 muestra ese reparto. El motor ILSA queda fuera de la extensión.

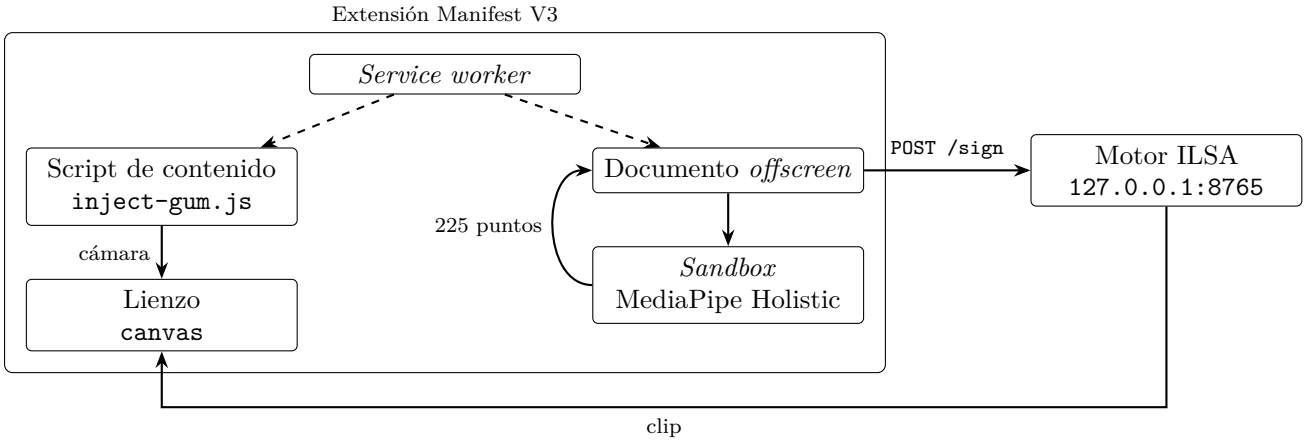


Figura 6.1: Paquetes de la extensión. La extracción corre en el *sandbox*. El lienzo recibe el clip ya armado.

6.3.1. Despliegue en Chrome Manifest V3: aislamiento offscreen y sandbox

El script en segundo plano coordina la mensajería entre el emergente de la extensión, la videollamada de Google Meet y el documento en segundo plano (Google Chrome Developers, 2024). Al detectar la carga activa de una videollamada, el script verifica la disponibilidad del servicio local ILSA en el puerto local (Ramírez, 2018). Al confirmar la conectividad, instancia el documento en segundo plano mediante la API `chrome.offscreen` (Google Chrome Developers, 2024).

Dentro de este esquema, el documento *offscreen* carga un marco iframe configurado como entorno aislado (*sandbox*) con políticas de seguridad relajadas (`Content-Security-Policy`), permitiendo la ejecución de código WebAssembly y la instanciación de los trabajadores auxiliares (*web workers*) de MediaPipe Holistic (Google, 2020; Lugaresi et al., 2019). Para reducir la carga de CPU, las imágenes de la cámara se escalan a 320×240 píxeles antes de transferirse mediante `postMessage` hacia el entorno aislado. La subrutina de visión calcula los 225 puntos articulares tridimensionales y devuelve las coordenadas vectoriales al documento *offscreen*, el cual consolida la ventana temporal de 16 fotogramas y despacha la petición al motor local (Google Chrome Developers, 2024).

6.3.2. Intercepción de la captura de cámara y lienzo virtual

La captura de video se resuelve mediante la inyección del script `inject-gum.js` en el contexto principal de la página web (*MAIN world*), el cual sobrescribe el método estándar `navigator.mediaDevices.getUserMedia` (W3C, 2024).

Cuando la videollamada solicita acceso a la cámara web, el script intercepta la llamada, obtiene el flujo directo del hardware y lo vincula a un elemento `<video>` oculto en el DOM (W3C, 2024). En paralelo, los fotogramas se redibujan sobre un elemento `<canvas>` virtual que opera a 24 cuadros por segundo. A través del método `canvas.captureStream()`, la extensión genera un flujo sintético de medios que se entrega a Google Meet como si fuera el dispositivo físico original (W3C, 2024).

Esta derivación permite superponer la capa visual de subtítulos en la franja inferior del lienzo sobre un fondo semitransparente antes de emitir la señal a la videollamada (W3C, 2024). De esta manera, el interlocutor oyente puede visualizar las traducciones directamente en la señal de video transmitida por WebRTC sin necesidad de instalar la extensión en su propio navegador (W3C, 2024). La renderización aplica un reflejo horizontal coincidente con la vista previa local y sostiene la visibilidad del subtítulo durante un intervalo de 8 segundos por oración.

La secuencia de eventos que rige la ejecución integrada del canal directo comprende los siguientes pasos:

1. **Intercepción de video y aislamiento:** El script inyectado redirige el flujo de la cámara física hacia el lienzo sintético `<canvas>` mediante `captureStream()` (W3C, 2024) y transmite los fotogramas comprimidos al documento *offscreen* (Google Chrome Developers, 2024).

2. **Extracción cinemática en Sandbox:** El entorno aislado procesa los fotogramas con MediaPipe Holistic y retorna los 225 puntos articulares tridimensionales de torso y manos (Bazarevsky et al., 2020; Google, 2020; Zhang et al., 2020).

3. **Clasificación temporal continua en el motor local (ILSA):** La extensión serializa la secuencia de 16 fotogramas (16×225) y la transmite mediante una petición HTTP POST al endpoint `/sign` del servidor local FastAPI (Ramírez, 2018).

4. **Inferencia local y filtrado heurístico (`RepeatGate`):** El clasificador *TinySkeletonClassifier* genera la predicción. La compuerta de validación evalúa la glosa según su categoría léxica (admitiendo repeticiones en dígitos, un máximo de dos en letras y descartando repeticiones inmediatas en vocabulario general) y actualiza el búfer acumulador (Gamma et al., 1994).

5. **Temporización y cierre de enunciado:** Al transcurrir un intervalo continuo de 4 segundos sin actividad gestual, el acumulador declara el cierre del enunciado y dispara la solicitud de traducción.

6. **Invocación al servicio semántico:** El motor local transmite la lista de glosas mediante una petición HTTP POST al endpoint `/translate` del servidor remoto en la nube (Ramírez, 2018).

7. **Emisión concurrente de resultados:** La oración traducida en español se retorna a la extensión para dibujarse como subtítulo sobre el lienzo virtual de Google Meet (W3C, 2024) y, en simultáneo, se envía al hilo secundario de síntesis de voz (`pyttsx3`) para su emisión audible hacia el oyente (Pressman & Maxim, 2014).

6.3.3. Motor local ILSA (FastAPI)

El servidor local se implementó en Python mediante el framework asíncrono FastAPI y se ejecuta sobre un servidor ASGI Uvicorn (Ramírez, 2018). El servicio se enlaza exclusivamente a la interfaz de bucle local (`127.0.0.1:8765`) para denegar conexiones externas no autorizadas.

Para cumplir con las políticas de seguridad del navegador, el middleware del servidor implementa reglas CORS que restringen los orígenes admitidos a `https://meet.google.com` y al identificador de la extensión (`chrome-extension://*`) (Google Chrome Developers, 2024; Ramírez, 2018). Asimismo, responde con la cabecera de acceso a red privada (`Access-Control-Allow-Private-Network: true`) requerida cuando una página HTTPS realiza peticiones hacia el dominio local (Google Chrome Developers, 2024).

La emisión de voz sintética se desacopló mediante un hilo de ejecución secundario que consume una cola secuencial de mensajes, evitando bloqueos en la atención de las solicitudes HTTP de la API (Pressman & Maxim, 2014). El motor local se empaquetó como un ejecutable independiente de Windows (`ILSA.exe`) utilizando PyInstaller, e incluye una interfaz gráfica desarrollada en Tkinter para permitir la selección de modo (Sordo u Oyente), la configuración de la mano dominante y el control de encendido del servicio.

6.4. Implementación del avatar

6.4.1. Tecnologías empleadas

El servicio de archivos es un proceso aparte del traductor. La imagen parte de Python 3.10 en su variante reducida. Dentro van el programa y la carpeta de poses. El código anterior de captura y clasificación no se copia. El directorio de trabajo del contenedor

es `/app`. El proceso arranca con el programa principal y lee el puerto de la variable de entorno. Si esa variable no existe, usa 8080.

La publicación se hizo en Cloud Run, con el nombre `tpose-server`, en la región `southamerica-east1` y en el proyecto `ilsa-511622`. El comando de despliegue parte de la raíz del repositorio, sube el código, construye la imagen y deja el servicio abierto sin token de usuario. El puerto declarado hacia la plataforma es 8080. El proceso, igual, lee el puerto que la plataforma inyecta al arrancar. Los nombres de la carpeta son ASCII. Un archivo con eñe o con tilde en el nombre hace fallar la importación del contenedor, aunque la imagen se construya bien.

El dibujo corre en el lienzo de la extensión de Meet. El motor local, el mismo FastAPI de la Sección anterior, arma la lista de fotogramas. La extensión no estima puntos y no pide archivos por su cuenta. La Figura 6.2 muestra la instancia con la que se probó el prototipo: una sola copia de cada servicio, sin el balanceador de la proyección de producción.

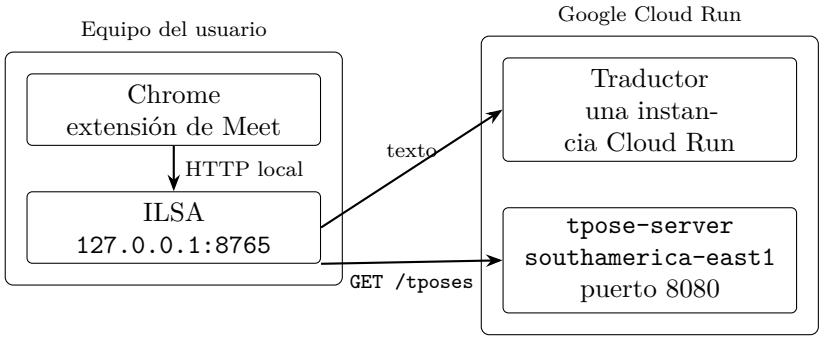


Figura 6.2: Despliegue del prototipo probado. El video no sale del equipo. Cada servicio remoto es una sola instancia.

6.4.2. Vía descartada: retarget hacia un avatar VRM

Antes del diccionario de poses se probó la primera familia de la Tabla 3.4: un personaje VRM, con malla y huesos de longitud fija, animado por rotaciones. El fundamento del ensayo es el *retarget*. MediaPipe Holistic entrega posiciones, y el avatar exige rotaciones sobre su propio esqueleto. El visor se armó con Three.js. Para la mano se probó Kalidokit, que resume cada dedo en un único ángulo de rizo, y una cinemática inversa que lleva el extremo del hueso hasta el punto medido.

La prueba no se retuvo. El desarrollo no contaba con práctica suficiente de animación tridimensional, y las bibliotecas de *retarget* no sostuvieron de forma estable el paso de la pose al esqueleto VRM. El emparejamiento fallaba con frecuencia porque la pose pierde la profundidad en el eje Z: la coordenada de la mano queda relativa a la muñeca y en una escala distinta del desplazamiento en el plano, de modo que el hueso no recibe una distancia utilizable. Si se respeta la longitud del hueso, la yema no cae sobre el punto medido. Si la cinemática inversa alarga el miembro hasta ese punto, la mano pierde la configuración y, en el alfabeto manual, la letra cambia. Kalidokit, además, confunde configuraciones próximas.

El diccionario de `.pose` evita ese paso: dibuja los puntos medidos y no los convierte en rotaciones de otro cuerpo. Si más adelante se retoma el avatar tridimensional, una línea de mejora es filmar cada seña desde tres ángulos, para reconstruir la profundidad que una sola toma frontal no conserva.

6.4.3. Estructura del pedido

ILSA, en modo oyente, no lee un diccionario de poses del disco de la videollamada. Después de obtener las glosas del canal inverso, normaliza cada nombre y pide `GET /tposes/{glosa}.pose`. `GET /health` solo indica que el proceso está arriba. Un 200 del archivo trae el binario. Un 404 indica que esa glosa no tiene clip.

En el motor local, `POST /hearing` hace las dos consultas: primero la oración hacia el canal inverso y después los archivos. La respuesta trae la oración, las glosas y el clip. `POST /avatar` recibe una cadena de glosas y repite solo la descarga. En modo señante las dos rutas responden 400. La dirección del servicio se reemplaza por configuración, sin recompilar el motor. Los archivos ya bajados quedan en la caché local del equipo. Una glosa repetida no vuelve a viajar por la red.

Si el nombre de una glosa no tiene archivo, ILSA no inventa el movimiento. Pide las letras de esa glosa al mismo servicio, cada una como un archivo. Si el servicio no responde, las glosas siguen disponibles como texto y el clip queda vacío. El tope de la cadena es 24 archivos.

6.4.4. Decisiones técnicas de la reproducción

La Sección 5.3 fija qué se muestra y por qué. Aquí quedan los valores con los que esas reglas se programaron, y cuál de los dos programas hace cada paso. La preparación del clip y el empalme entre glosas corren en ILSA, una vez que los archivos ya llegaron del servicio de poses. La extensión de la videollamada no rehace ese empalme. Recibe la lista de fotogramas ya armada y la dibuja en la mitad derecha del lienzo. El navegador, entonces, no vuelve a calcular el puente: elige, según el tiempo transcurrido, cuál de esos fotogramas le toca mostrar.

Valores del compositor. Antes del recorte, todos los clips se llevan a 30 fotogramas por segundo. Con esa tasa común, una espera de seis fotogramas dura lo mismo en cualquier seña. Si se dejara cada video a la cadencia con la que fue filmado, seis fotogramas de un clip a 24 por segundo durarían más que seis fotogramas de un clip a 30, y el empalme perdería la medida del tiempo.

El suavizado es un promedio exponencial. Si x_t es la posición recién medida y p_{t-1} es la pose ya aceptada en el fotograma anterior, la pose que se conserva es

$$p_t = \alpha\, x_t + (1 - \alpha)\, p_{t-1}, \tag{6.2}$$

con $\alpha = 0{,}50$. La medición nueva y la pose anterior pesan lo mismo. Ese peso es único para todo el diccionario.

El recuadro común mide 720×780 píxeles. El cuerpo se escala para que el ancho de hombros quede en 250 píxeles. Así, dos tomas de distinto encuadre ocupan el mismo espacio cuando se pegan una detrás de la otra.

El puente dura entre 4 y 8 fotogramas. Si las muñecas del final de una seña y del inicio de la siguiente ya están cerca, se usan 4. Si están lejos, se llega a 8, para que la mano no salte. Un punto cuya confianza queda por debajo de 0,15 sale en la lista como ausente, y el trazo del lienzo no lo une con el punto anterior.

La entrada, desde la postura de espera hasta el primer fotograma de la seña, y la salida, de vuelta a esa espera, duran cerca de seis fotogramas de la tasa de 30. A esa cadencia, cada uno de esos tramos ocupa cerca de un quinto de segundo. La secuencia se corta en el techo fijado en el diseño: 24 glosas y 1200 fotogramas.

Postura de espera. No hay un archivo aparte llamado reposo. El cuerpo y las manos de la espera se copian de los primeros fotogramas de una seña de referencia, el tramo en el que la persona todavía no empezó el gesto, y se escalan al mismo ancho de hombros del lienzo. El antebrazo queda casi horizontal, levemente hacia abajo, y la mano sigue esa línea.

Reflejo de la miniatura local. La videollamada muestra la miniatura de quien usa el sistema como un reflejo. La extensión deshace ese reflejo solo en ese video local. El esqueleto que viaja hacia la otra persona no se invierte. Las dos vistas quedan iguales: la cámara en la mitad izquierda y el esqueleto, sin darlo vuelta, en la mitad derecha.

6.5. Instancia de validación del prototipo mediante túnel seguro

La validación funcional del microservicio semántico remoto se realizó mediante un túnel seguro cifrado (*ngrok*). Esta instancia conectó el motor ILSA local con un servidor semántico desacoplado montado en una máquina independiente, demostrando la viabilidad de la partición híbrida sobre redes residenciales reales sin incurrir en costos de infraestructura elástica comercial.

Capítulo 7

Experimentación, Resultados y Discusión

7.1. Corpus y metodología de adquisición de Datos de LSA

El clasificador no se entrena sobre un corpus público de otra lengua de señas. Un conjunto de prueba en ASL se descartó en las primeras iteraciones: el léxico, la gramática y, en varios ítems, la forma de la seña no coinciden con la LSA, y ese material presentaba además etiquetas inconsistentes y videos espejados. El antecedente local LSA64 aporta un protocolo de captura controlada, con guantes de colores, que no se reproduce en una cámara web doméstica (Ronchetti et al., 2016). El entrenamiento usa, por eso, un corpus propio de LSA.

7.1.1. Definición y validación del vocabulario de 100 clases

El vocabulario objetivo cubre cerca de 100 señas, definidas con la orientación de Miriam Rolls. El recorte léxico apunta a dos situaciones de uso: identificación y verbos de emergencia, por un lado, y asistencia al viajero (lugares, días y familia), por el otro. No se pretende cubrir una conversación abierta.

El entrenamiento no incorpora las 100 señas de manera automática. Una clase entra al modelo solo si alcanza un piso de videos. En julio de 2026 el piso era de 50 videos y el modelo activo tenía 91 clases. En agosto el piso pasó a 60 videos por clase y el conjunto activo llegó a 94, con el alta de `chau`, `tener` y `años`. En septiembre el checkpoint retiene 97 clases: a las 94 se suman `pasado`, `futuro` y `mio`. La letra `ñ` ya formaba parte del conjunto de julio.

Las 97 glosas se agrupan en interrogativos, afirmación y negación, dígitos, alfabeto dactilológico, pronombres, saludos, tiempo, parentesco, lugares y un conjunto cerrado de verbos. El piso de muestras deja fuera del checkpoint a las señas del léxico objetivo que no llegan a 60 videos. Esa exclusión es deliberada: con pocos clips el modelo memoriza al señante de la grabación y la matriz de confusión deja de ser interpretable.

7.1.2. Protocolo de grabación y curación del dataset propio

Los videos de entrenamiento están en 1920×1080 (16:9). La ficha de septiembre contabiliza 6 748 archivos MP4, cerca de 69 clips por seña, y 6 680 tensores `.npy` de forma

(16, 225) en `dataset_landmarks_16frames`. El entrenamiento no usa todos los clips de una clase. Con semilla 42 se mezclan los archivos, se retienen como máximo 60 por clase y el corte es 80 % entrenamiento y 20 % validación, es decir 48 y 12 clips por clase en el régimen de 60 muestras. En el checkpoint del 15/09/2026 eso da 4 656 secuencias de entrenamiento y 1 164 de validación.

El corte es por clip, no por persona. Clips de la misma sesión pueden caer a ambos lados del split. La exactitud de validación queda, por eso, por encima de lo que después mide la cámara: el conjunto de validación no es un señante nuevo ni una óptica nueva.

La aumentación opera solo en entrenamiento y no se guarda en disco. A la escala isotrópica (0,85–1,15) y al ruido gaussiano se suman, desde agosto, rotación 3D (yaw $\pm 15°$, pitch y roll $\pm 8°$), recorte temporal de hasta 15 % por extremo, deformación de velocidad entre 0,80 y 1,25, anulación de hasta 3 fotogramas con interpolación posterior, y ruido distinto en pose (0,012) y en manos. Un multiplicador virtual de 25 repite cada clip con una transformación nueva en cada época. No hay espejado horizontal aleatorio. La validación no recibe aumentación ni repetición virtual.

Dos regrabaciones quedaron incompletas y condicionan los errores finales. La letra `V` se volvió a grabar el 08/09/2026 como pose estática (65 videos de esa fecha). La seña `viernes`, que consiste en la misma configuración de mano con dos golpes al mentón en el eje z, conserva tomas hasta el 17/08/2026 y no se regrabó. El par sigue ambiguo en el dato.

7.2. Evaluación empírica del clasificador gestual

La selección del modelo no sigue la exactitud de validación. El criterio de integración es el acierto top-1 y top-3 frente a la cámara, con una toma por seña, mano diestra y modo `auto`. A lo largo de julio, agosto y septiembre ese criterio descartó la red de mayor capacidad, la búsqueda automática de hiperparámetros y una línea paralela de *transfer learning*, y retuvo el baseline de 16 fotogramas.

7.2.1. Métricas offline vs. rendimiento en cámara real

La Tabla 7.1 resume la brecha que se repite en cada iteración. La validación, sobre clips del mismo corpus y del mismo extractor, supera el 96 %. La cámara, con otra óptica, otra luz y una sola toma, cae entre 15 y 35 puntos según la sesión.

Cuadro 7.1: Exactitud de validación y acierto en cámara del clasificador. Top-1 y top-3 de cámara son crudos: no colapsan homógrafos.

Checkpoint	**Fecha**	**Val. top-1**	**Cámara top-1**	**Cámara top-3**
16 fotogramas, 91 clases	29–30/07	97,36 %	61,5 %	83,5 %
32 fotogramas, 128/4/2	31/07	96,48 %	71,4 %	87,9 %
32 fotogramas, 64/2/1	31/07	97,36 %	67,0 %	90,1 %
Optuna, 32 fotogramas	31/07	97,69 %	66,0 %	84,6 %
Baseline 16f, luz baja, 94 clases	18/08	98,40 %	67,0 %	89,4 %
Baseline 16f, luz alta, 94 clases	28/08	98,40 %	86,2 %	95,7 %
Baseline 16f, 97 clases	15–17/09	98,28 %	77,3 %	94,8 %

Tres factores explican la caída, y no son intercambiables. El primero es el dominio: la

validación reutiliza el extractor y el encuadre de los MP4 de entrenamiento. La cámara de desarrollo no. El segundo es la luz. El mismo baseline de 16 fotogramas pasa de 67,0 % top-1 el 18/08, con luz ambiente, a 86,2 % el 28/08, con mejor iluminación. Ese salto no proviene de un cambio de pesos. El tercero es la varianza de una sola toma. Dos pasadas del mismo conjunto de 36 letras y números, sin cambio de pipeline, movieron el top-1 entre 11 y 19 puntos según el checkpoint. Con una toma por seña, la cámara no distingue efectos menores a unos 14 puntos. Por eso la corrección de relación de aspecto se midió también fuera de línea, sobre los mismos tensores, y no solo con una segunda sesión en vivo.

Una línea paralela de *transfer learning* sobre landmarks alcanzó exactitudes de 0,98 y 0,99 en reportes de 1 084 muestras. Esos reportes incluyen clases de soporte nulo y no cierran el protocolo de cámara que definió el modelo integrado. El checkpoint que se retiene es el TinySkeleton entrenado desde cero.

7.2.2. Impacto de la resolución temporal: 16 vs. 32 fotogramas

En julio, con 91 clases y 50 videos por clase, se compararon 16 y 32 fotogramas bajo el mismo pipeline de recorte y submuestreo. La hipótesis era que más fotogramas conservarían mejor los gestos largos. La Tabla 7.2 y las Figuras 7.1 y 7.2 resumen el resultado.

Cuadro 7.2: Resolución temporal en julio de 2026 (91 clases, 50 videos por clase).

Iteración	**Arquitectura**	**Val. top-1**	**Cámara top-1**	**Cámara top-3**
A, 16 fotogramas	128 / 4 cabezas / 2 capas	97,36 %	61,5 %	83,5 %
B, 32 fotogramas	128 / 4 cabezas / 2 capas	96,48 %	71,4 %	87,9 %
C, 32 fotogramas	64 / 2 cabezas / 1 capa	97,36 %	67,0 %	90,1 %

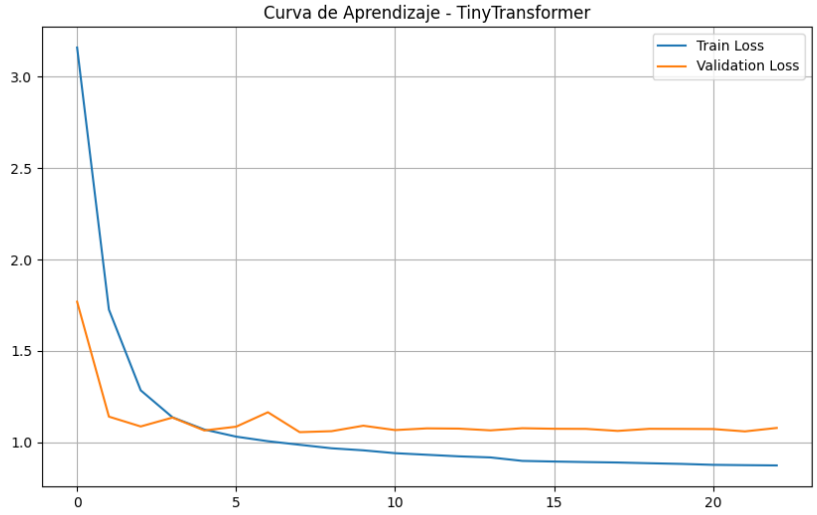


Figura 7.1: Curva de aprendizaje de la iteración A (16 fotogramas, 29/07/2026). La pérdida de validación se estanca cerca de 1,05 desde la época 7, mientras la de entrenamiento sigue en descenso.

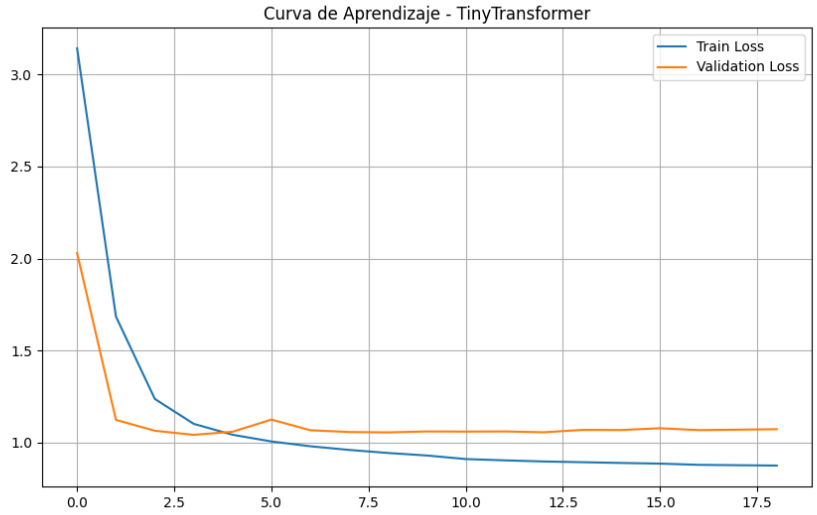


Figura 7.2: Curva de aprendizaje de la iteración B (32 fotogramas, 128 dimensiones, 4 cabezas y 2 capas, 31/07/2026). La pérdida de validación se estabiliza cerca de 1,04 desde la época 4.

En esa semana, 32 fotogramas suman 9,9 puntos de top-1 en cámara respecto de 16 (71,4 % contra 61,5 %). La red más chica de 32 fotogramas no mejora el top-1 y sí el top-3, que llega a 90,1 %. La exactitud de validación, en cambio, casi no se mueve. Más contexto temporal ayuda en vivo con el corpus de julio, y no cierra la brecha con la validación.

A partir de agosto el corpus pasa a 60 videos por clase, la aumentación se amplía y la comparación en cámara, con luz pareja, favorece otra vez a 16 fotogramas. El baseline de 16 fotogramas, 128 dimensiones, 4 cabezas y 2 capas supera en top-1 a las variantes de 12 y de 10 fotogramas. El modelo integrado vuelve a 16 fotogramas. La lectura de julio no se anula: con poco dato, alargar la secuencia mejoró la cámara. Con más dato y con el criterio de producto (top-1 en vivo, letras y dígitos incluidos), la secuencia corta generaliza mejor y evita fotogramas repetidos en las señas estáticas.

7.2.3. Búsqueda automatizada de hiperparámetros con Optuna y Análisis de sobreajuste

Optuna se usó en dos rondas, siempre con validación cruzada de 3 pliegues sobre el conjunto de entrenamiento y con poda de ensayos pobres (Akiba et al., 2019). El objetivo del estudio es la pérdida de validación, no el acierto en cámara.

La ronda de julio, sobre 32 fotogramas y 40 ensayos, elige 128 dimensiones, 8 cabezas, 2 capas, dropout 0,30, lote de 16 y una tasa de aprendizaje de $3{,}77 \times 10^{-5}$. El reentrenamiento con esos valores baja la pérdida de validación a 0,087 y sube la exactitud offline a 97,69 %. En cámara, la corrida oficial da 66,0 % top-1 y 84,6 % top-3, por debajo de la iteración manual B en ambas métricas. Una primera pasada de esa noche se descartó porque una seña se cortó al salir la mano del cuadro.

La ronda de agosto (estudio `tinyskeleton_lsa_v2`, 40 ensayos) libera también la longitud temporal, con candidatos de 8, 12 y 16 fotogramas. El mejor ensayo queda en 12 fotogramas, 256 dimensiones, 2 cabezas, 2 capas y dropout 0,57, con pérdida media de validación 0,137. En cámara, con la misma luz que el baseline del 28/08, ese checkpoint

rinde 80,9 % top-1 crudo y 97,9 % top-3. El baseline de 16 fotogramas, en esa sesión, rinde 86,2 % top-1 y 95,7 % top-3. Optuna gana cerca de 2 puntos de top-3 y pierde cerca de 4 de top-1, sobre todo en letras. Una tercera red, de 10 fotogramas, 256 dimensiones y 3 capas, alcanza 99,02 % de exactitud offline y 77,7 % top-1 en cámara. Una evaluación apareada del 31/08, con la misma toma inferida en las tres redes, confirma el orden: el baseline queda primero en top-1 y también en top-3. La red de 3 capas dispara confianzas cercanas a 1,0 en errores.

El sobreajuste de las curvas de julio es visible en las Figuras 7.1 y 7.2: la pérdida de entrenamiento sigue en descenso y la de validación se estanca por encima de 1. Ese patrón se atenúa en los reentrenamientos de agosto y septiembre, con más dato, más aumentación y parada temprana. La curva final (Figura 7.3) desciende en ambos conjuntos y mantiene una brecha chica. La falla que permanece no es una divergencia de la curva offline. Es el cambio de dominio hacia la cámara.

7.2.4. Diagnóstico de confusiones mediante feature engineering y separabilidad

Con la arquitectura ya elegida, la iteración de septiembre no agrega capacidad a la red. Construye un descriptor de 26 medidas, calculado al vuelo sobre el vector de 225 dimensiones, para responder dos preguntas: qué pares están geométricamente juntos, y si un fallo de cámara también existe en el corpus.

El descriptor tiene tres familias. Seis medidas de geometría mano–rostro, en anchos de hombro, distinguen el lugar de apoyo. Seis medidas de trayectoria de muñeca (desvío y rango en x, y y z) distinguen una letra estática de la seña dinámica que reutiliza esa forma de mano. Catorce medidas de configuración de dedos (extensión, curvatura y separación), normalizadas por el largo de la mano y no por los hombros, distinguen formas que ocupan el mismo lugar y casi no se mueven. Estas 14 se agregaron después de comprobar que las 12 primeras eran ciegas a la forma de la mano.

El descriptor no se concatena al tensor de entrenamiento. El espejado, la aumentación 3D y el recorte temporal asumen que todo el vector son ternas (x, y, z). Intercalar 26 escalares rompería esas operaciones. El cálculo queda fuera del `.npy` y no entra al modelo en esta iteración.

La separabilidad de un par se mide con el AUC de Mann–Whitney sobre cada feature (Mann & Whitney, 1947). El índice informado es $\text{sep} = |\text{AUC} - 0{,}5| \times 2$: vale 1 cuando las dos distribuciones no se solapan y 0 cuando coinciden. En paralelo, un 1-NN con exclusión del propio clip, sobre el tensor $(16, 225)$ aplanado y sobre 6 680 clips, alcanza 88,4 % de acierto. Ese número es optimista, porque clips de la misma sesión se parecen entre sí. Lo útil es qué pares se cruzan.

Los pares duros del alfabeto y de los dígitos se separan con las features de dedos: `2` y `5` por la separación entre medio y anular (sep = 1,00), `D` y `O` por la curvatura del índice (0,97), `M` y `N` por la curvatura del meñique (1,00). El control negativo es `O` y `O`, que permanece en 0,53: comparten forma y movimiento, y ninguna medida del descriptor los separa. `V` y `viernes` quedan en 0,28 de movimiento, coherente con la regrabación incompleta de `viernes`.

El cruce con la cámara del 17/09 marca la mayoría de los fallos como ejecución o captura, no como ausencia de señal en el corpus. `lunes` frente a `L`, `donde` frente a `F` y `años` frente a `G` se separan en los `.npy`. Cuando en vivo la seña dinámica se ejecuta sin el agite, el modelo emite la letra. El descriptor sirvió para decidir qué regrabar y qué no cablear a

la red. No se incorporó al tensor.

7.2.5. Impacto cuantitativo de la corrección de aspect ratio

El efecto del relleno 16:9 se midió primero fuera de línea, sobre 36 tomas ya grabadas en 4:3 y sobre el checkpoint del 29/08. Multiplicar la coordenada y por 4/3 es la corrección analítica de la distorsión. La Tabla 7.3 muestra el resultado, determinista y sobre las mismas tomas.

Cuadro 7.3: Corrección de relación de aspecto sobre 36 letras y números ya grabados (checkpoint del 29/08).

Variante	**Top-1**
Captura original 4:3	66,7 %
Corrección solo en y ($\times$1,333)	88,9 %
Corrección en y y en z	75,0 %

La corrección en y suma 22,2 puntos. Ocho de los diez cambios de predicción van hacia la clase correcta. El caso extremo es `I`: pasaba a `ojo` con confianza 1,000 y pasa a `I` con confianza 1,000. Corregir también z empeora el resultado, en línea con la medición de que el relleno no debe alterar ese eje. De los 12 fallos originales quedan cuatro: `E`, `G`, `O` y `V`. `O` es el homógrafo real con `0`. `V` correspondía a la grabación anterior, ya reemplazada en el reentrenamiento de septiembre.

La variante sin corrección reproduce el 66,7 % de la sesión en vivo de ese modelo. La carga de pesos y el armado del tensor coinciden entre la cámara y la inferencia sobre `.npy`. La divergencia que todavía aparezca en otro cliente queda localizada en la extracción de landmarks, no en la red.

En la evaluación del 17/09, ya con relleno activo (640 $\times$ 480 a 853 $\times$ 480) y con el baseline de 97 clases, `I` sale `I` con confianza 0,89. En las dos pasadas del 15/09, sin relleno, los tres modelos la enviaban a `ojo` con confianza cercana a 1. `T`, `E` y `G` también aciertan en la sesión con relleno. El 88,9 % de las 36 letras y números coincide en valor con una pasada previa en 4:3, pero no es el mismo resultado: se gana `I` y se pierde `V`, porque el vocabulario ahora contrasta esa letra con `viernes`.

7.2.6. Desempeño final del clasificador baseline en vivo

El modelo que se integra es el baseline de 16 fotogramas, 128 dimensiones, 4 cabezas, 2 capas y dropout 0,4, con 97 clases. El checkpoint del 15/09/2026, entrenado 35 épocas con el extractor de Python, alcanza 98,28 % de exactitud top-1 en validación y una pérdida de 0,062. La evaluación en cámara del 17/09, una toma por seña, MediaPipe 0.10.21 y relleno 16:9, da 75 aciertos sobre 97 (77,3 % top-1) y 92 sobre 97 (94,8 % top-3). Cinco señas quedan fuera del top-3: `donde`, `ellos`, `familia`, `nosotros` y `repetir`. Las 36 letras y números llegan a 88,9 % top-1 y 100 % top-3. Si se acepta el homógrafo `O`$\equiv$`0` como acierto, el top-1 de las 97 sube a 78,4 %. Colapsar además las parejas letra–seña dinámica llevaría el índice a 83,5 %, y ese colapso no es el protocolo objetivo: el sistema debe separarlas.

La confianza media es 0,971 en los aciertos y 0,833 en los fallos. El modelo se equivoca con seguridad alta. Un umbral de 0,75 no filtra la mayor parte de esos errores.

Un reentrenamiento del 21/09/2026 conserva la misma arquitectura y el mismo vocabulario, y cambia el extractor al build de Holistic en navegador. Corre 32 épocas, con pérdida

de validación 0,052 y exactitud top-1 de 99,14 %. Las Figuras 7.3 y 7.4 corresponden a esa corrida, que es la última curva y la última matriz archivadas. La sesión de cámara del 17/09 no evalúa estos pesos: evalúa el checkpoint de Python del 15/09 (98,28 %). La matriz de la Figura 7.4 es casi diagonal porque describe la validación offline, no los fallos de cámara.

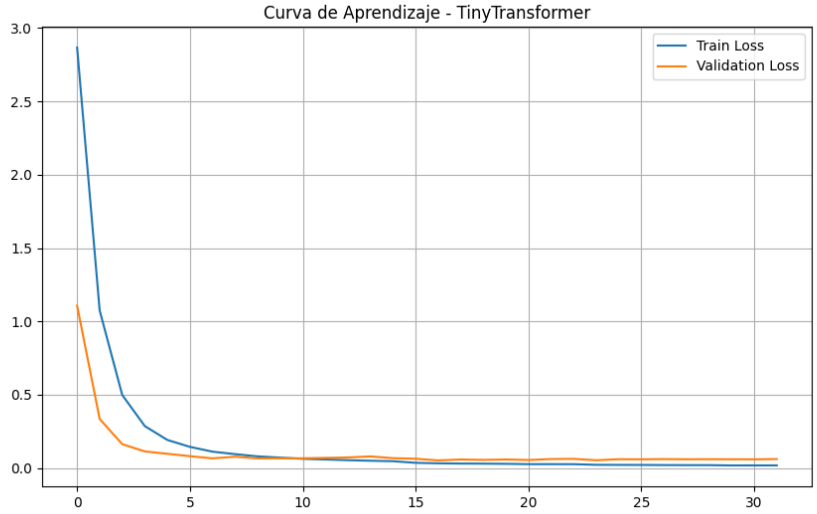


Figura 7.3: Curva de aprendizaje del reentrenamiento del 21/09/2026: 16 fotogramas, 128 dimensiones, 4 cabezas, 2 capas y 97 clases. La pérdida de validación mínima archivada es 0,052 (exactitud top-1 de 99,14 %).

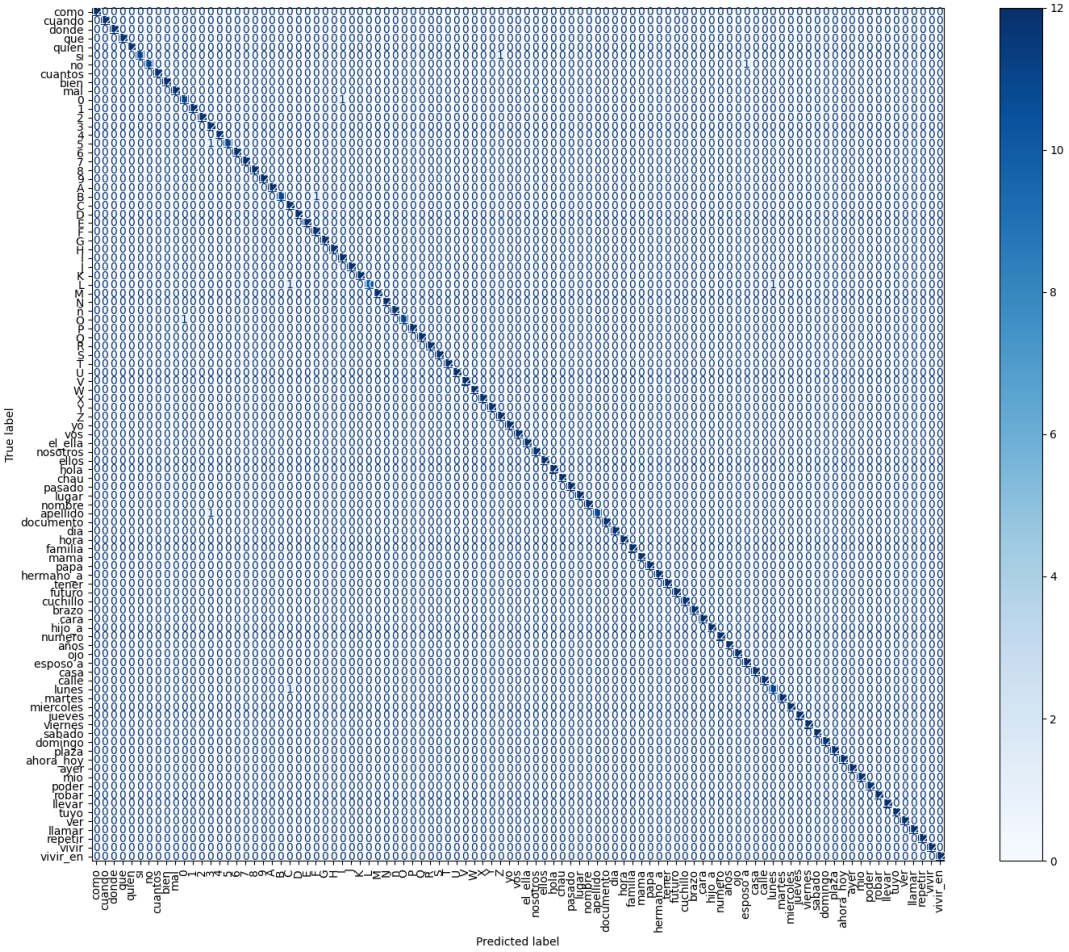


Figura 7.4: Matriz de confusión en validación del mismo reentrenamiento (21/09/2026, 97 clases). La masa se concentra en la diagonal. Los errores de cámara del 17/09 no aparecen en esta figura.

Los bloqueos que la sesión del 17/09 deja abiertos son cuatro. `V` y `viernes` se intercambian en ambos sentidos, porque `viernes` no se regrabó. `O` y `O` siguen como el mismo gesto para el dato disponible (sep = 0,53). Las señas dinámicas que se ejecutan sin agite caen en la letra correspondiente, aunque el corpus sí las separa. `pasado`, clase nueva, atrae a `ayer` y a `cuando`. Diecisiete de los 22 fallos quedan clasificados como captura o ejecución, dos como ambigüedad de `V`/`viernes` y tres en el límite (`O`/`O`, `años`/`G` y `ojo` frente a `vivir`, este último con confianza 0,53 contra 0,47).

Sobre las 94 señas en común con la evaluación del 31/08, el baseline de agosto tenía 86,2 % y esta corrida 77,7 %. El descenso no se atribuye a la red en forma aislada: cambian el vocabulario, la relación de aspecto de la captura y la letra `V`. El cambio atribuible al relleno es la recuperación de `I` y de `T`. El siguiente paso de dato, no de arquitectura, es regrabar `viernes` y repetir las tomas frágiles. Con una sola toma, el porcentaje global sigue sujeto a la varianza ya medida.

7.3. Evaluación de los módulos de traducción semántica

7.3.1. Benchmark comparativo de modelos SLM (Familias Qwen2.5, Llama 3.2 y SmolLM2)

7.3.2. Evaluación cuantitativa: BLEU, ROUGE-L, METEOR y exactitud

7.3.3. Análisis de simetría en el aprendizaje: traducción directa vs. estructuración inversa

7.3.4. Compromiso entre calidad lingüística y latencia de inferencia

7.4. Evaluación del sistema integrado y rendimiento en tiempo real

7.4.1. Validación funcional de extremo a extremo en videollamadas reales

7.4.2. Análisis de latencias totales por etapa del pipeline

7.4.3. Consumo de recursos de hardware: máquina de desarrollo vs. máquina objetivo

7.4.4. Discusión técnica y lecciones de ingeniería aprendidas

Capítulo 8

Conclusiones y trabajo futuro

8.1. Conclusiones

8.1.1. Cumplimiento de objetivos del proyecto

8.1.2. Aportes técnicos y de accesibilidad

8.1.3. Impacto social en el marco de la Ley 27.710

8.2. Trabajo futuro

8.2.1. Implementación física del clúster semántico escalable

8.2.2. Expansión del corpus multiseñante y mitigación de variabilidad

8.2.3. Incorporación de rasgos no manuales (expresiones faciales)

8.2.4. Generalización a nuevas plataformas de comunicación virtual

Bibliografía

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework, 2623-2631.

Alibaba Cloud. (2024). Qwen2.5: A Family of Large Language Models.

Allal, L. B., Lozhkov, A., Bakouch, E., von Werra, L., & Wolf, T. (2024). SmolLM2: When Small Goes Big.

Barán, V. (2018). LSApp: Uniendo dos mundos.

Bazarevsky, V., Grishchenko, I., Raveendran, K., Zhu, T., Zhang, F., & Grundmann, M. (2020). BlazePose: On-device Real-time Body Pose Tracking.

Blessit. (2024). Interpretación Lengua de Señas — Central de intérpretes.

Confederación Argentina de Sordos. (2024). Señario de términos y expresiones en Lengua de Señas Argentina.

Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., et al. (2024). The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.

Frantar, E., Ashkboos, S., Torfi, A., & Alistarh, D. (2022). GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *arXiv preprint arXiv:2210.17323*.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Gerganov, G. (2023). *llama.cpp: Port of Facebook's LLaMA model in C/C++*. https://github.com/ggerganov/llama.cpp

Google. (2020). MediaPipe: A Framework for Building Perception Pipelines.

Google Chrome Developers. (2024). Overview of Manifest V3.

Hand Talk. (2025). Acessibilidade digital em línguas de sinais.

HETAH. (2024). Avatar traductor a lengua de señas.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *Proceedings of the International Conference on Learning Representations*.

Hugging Face. (2024). Transformers: State-of-the-Art Machine Learning for PyTorch, TensorFlow, and JAX.

INJU. (2024). Diccionario de Señas.

Li, D., Rodriguez, C., Yu, X., & Li, H. (2020). Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods Comparison. *Proceedings of the IEEE Winter Conference on Applications of Computer Vision (WACV)*, 1459-1469.

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., & Grundmann, M. (2019). MediaPipe: A Framework for Building Perception Pipelines. *Third Workshop on Computer Vision for AR/VR at IEEE CVPR*.

Mann, H. B., & Whitney, D. R. (1947). On a Test of Whether One of Two Random Variables is Stochastically Larger than the Other. *The Annals of Mathematical Statistics*, *18*(1), 50-60.

Massone, M. I. (1993). El número y el género en la Lengua de Señas Argentina. *Signo & Seña*, *2*, 73-99.

Massone, M. I., & Machado, E. M. (1993). *Lengua de Señas Argentina: Análisis y Vocabulario Bilingüe*. Edicial.

Massone, M. I., & Martínez, R. A. (2012). *Curso de Lengua de Señas Argentina: Parte 1 y 2*. Cultura Sorda. http://www.cultura-sorda.eu

Mindlin, I. (2021). *Reconocimiento de Lengua de Señas con redes neuronales recurrentes* [Tesis de grado]. Universidad Nacional de La Plata, Facultad de Informática. http://sedici.unlp.edu.ar/handle/10915/129853

Ministerio de Justicia de la Nación. (2024). Características de la comunicación por videollamadas — Línea 144.

Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Levskaya, A., Zhang, C., Nado, Z., Jaitly, N., Vaswani, A., et al. (2023). Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, *5*, 606-624.

Pressman, R. S., & Maxim, B. R. (2014). *Software Engineering: A Practitioner's Approach* (8th). McGraw-Hill.

Ramírez, S. (2018). *FastAPI: Modern, fast (high-performance), web framework for building APIs with Python*. https://github.com/fastapi/fastapi

República Argentina. (2023). Ley 27.710 — Lengua de Señas Argentina.

Ronchetti, F., Quiroga, F., Estrebou, C., Lanzarini, L., & Rosete, A. (2016). LSA64: An Argentinian Sign Language Dataset. *XXII Congreso Argentino de Ciencias de la Computación (CACIC)*, 794-803.

SignAll Technologies. (2026). About SignAll Technologies.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *arXiv preprint arXiv:1706.03762*.

W3C. (2024). Media Capture from DOM Elements: `HTMLMediaElement.captureStream()`.

Wu, Y., Lu, S., Gan, Z., Gan, C., Shen, Y., Chen, Y.-C., Smith, L., Ren, S., Carin, L., He, X., et al. (2020). Lite Transformer with Long-Short Range Attention. *arXiv preprint arXiv:2004.11886*.

Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., & Grundmann, M. (2020). MediaPipe Hands: On-device Real-time Hand Tracking.